

VR-Nexus Project Documentation

Backend, Frontend, React Structure and Viva Q&A

Muhammad Afnan Khan

September 2026

Contents

VR-Nexus Backend, Explained	2
1. What this backend does, in one request	2
2. Core concepts for beginners	2
3. File-by-file walkthrough	5
4. Patterns worth reusing in your own projects	22
VR-Nexus Frontend, Explained	24
Core concepts for beginners	24
File-by-file walkthrough	26
Patterns worth reusing	38
React Project Structure and Workflow, Taught Through VR-Nexus	40
1. The folder tree	40
2. What each folder is for, and why it stays separate	41
3. How a request flows end to end	43
4. How routing and layouts nest	45
5. State management map	46
6. Build and dev tooling, briefly	47
7. Checklist for starting a new React project from scratch	47
VR-Nexus Viva Preparation: Questions and Answers	49
1. Project Overview and Motivation	49
2. High-Level Architecture	49
3. Tech Stack Justification	50
4. Security Design Decisions	52
5. The Tender Analysis Pipeline End to End	53
6. The Evidence Library and RAG Pipeline End to End	54
7. Database Design	55
8. Frontend Architecture	56
9. Testing, Reliability, and Failure Modes	57
10. Tradeoffs and Honest Limitations	57
11. Future Work: "If You Had More Time"	58
12. Rapid-Fire Reference	59

VR-Nexus Backend, Explained

This document walks through the entire backend of VR-Nexus, a Tender Intelligence and Evidence Matching Platform, file by file. It is written for a final-year Computer Systems Engineering student who knows Python but has not built with FastAPI, SQLAlchemy, Celery or Redis before. The goal is twofold: understand this specific codebase well enough to extend it safely, and pick up patterns you can reuse in other full-stack projects.

1. What this backend does, in one request

VR-Nexus has two pipelines. The Tender Analysis pipeline takes an uploaded procurement tender (a PDF, often hundreds of pages), extracts every requirement it contains using an LLM, matches each requirement against a company's evidence library using vector search, and produces a reviewable Excel tracker plus a downloadable output folder. The Evidence Library pipeline takes a company's own documents (case studies, methodology write-ups, certificates, company profiles), parses them, splits them into chunks, embeds those chunks into vectors, and makes them searchable, both for the tender-matching step and for a "grounded Ask" feature where a user can ask a question and get an answer built only from retrieved passages, with citations.

A single request through this backend generally follows the same shape. A client (the React frontend) sends an HTTP request to a FastAPI route. FastAPI first runs the route's declared dependencies, most importantly `get_current_user`, which reads the `Authorization: Bearer <token>` header, decodes and verifies the JWT, and loads the corresponding User row from Postgres, rejecting the request with 401 if any of that fails. If the request carries a body, FastAPI validates it against a Pydantic schema before your route function ever sees it, so the route can assume the data is already well-shaped. The route function then talks to the database through a SQLAlchemy Session, which is itself provided as a dependency (`get_db`) and is closed automatically once the request finishes. For anything that takes more than a moment (parsing a 300-page tender, calling an LLM 150 times, running OCR, computing embeddings), the route does not do the work itself. It writes a row describing the job, queues a Celery task, and returns immediately, often in well under a second. A worker process, running Celery separately from the web server, picks up that task, does the slow work, and as it progresses publishes small JSON progress messages to Redis. The frontend, meanwhile, has opened a WebSocket connection (authenticated with a short-lived ticket rather than the JWT itself) that is subscribed to those Redis messages, so the browser sees the tender move through "Parsing", "Chunking", "Extracting requirements", "Matching evidence" and so on in near real time, without ever polling.

2. Core concepts for beginners

If you have not used any of FastAPI, SQLAlchemy, Alembic, Celery, Redis, WebSockets, JWTs or RAG before, read this section before the file-by-file walkthrough. Everything below is explained in plain terms with the goal of transferring, not just describing this one codebase.

FastAPI path operations. A "path operation" is just a Python function decorated with something like `@router.post("/api/tenders")` that FastAPI calls when a matching HTTP request arrives. Think of it as a named entry in a big dispatch table: the framework reads the URL and method off the incoming request, looks up which function you registered for that combination, and calls it, passing in whatever the function asked for (a validated request body, a path parameter, a database session, the current user) as ordinary Python arguments. You never manually parse the request; you declare what you want in the function signature, and FastAPI's own machinery figures out how to produce it. In this codebase every path operation lives in a module under `app/api/routes/`, each module owns one `APIRouter` with its own URL prefix, and `app/main.py` collects all of those routers onto one `FastAPI()` app.

Pydantic schemas versus SQLAlchemy models. These are two different jobs even though they can look similar (both are “just a class with fields”). A Pydantic schema, defined under `app/schemas/`, describes what crosses the network wire: what a request body must look like to be accepted, and what shape a response will be serialized into as JSON. A SQLAlchemy model, defined under `app/models/`, describes a row in the database: which table it lives in, which columns it has, what type each column is, how it relates to other tables. The two are kept apart on purpose. A database row can carry things a client should never see (an internal Celery task id, a raw file path on the server’s disk) and a client request can carry things that never touch the database directly (a raw comma-separated keywords string that gets split into a list before storage). If you collapsed the two into one class, every change to your table would leak into your API and every change you want in your API would force a migration. Keeping them separate means the wire format and the storage format can evolve independently, and this codebase’s `DocumentOut/Document` and `TenderOut/Tender` pairs are a clean example of that split in practice.

ORM models. ORM stands for Object-Relational Mapper. Instead of writing raw SQL strings (`INSERT INTO users (...) VALUES (...)`), you write and manipulate ordinary Python objects, and the library translates your object operations into SQL for you. `db.add(user)` followed by `db.commit()` produces an `INSERT`. `db.get(User, some_id)` produces a `SELECT ... WHERE id = ...`. Relationships between tables (a `Tender` has many `Requirement` rows) become Python attributes you can just walk (`tender.requirements`). The benefit is that your business logic reads like normal Python and the ORM keeps the SQL, the escaping, and a lot of the book-keeping out of your way; the cost is that you need to understand roughly what SQL your ORM calls will generate, or you can write code that looks fine and performs terribly (loading a relationship one row at a time inside a loop, for instance, which is why you will see `selectinload(...)` used deliberately in a few places in this codebase to fetch related rows in one extra query instead of many).

Dependencies (`Depends()`). FastAPI’s dependency system is its answer to “how do I share setup logic across many endpoints without copy-pasting it into every function.” You write a small function, say `get_current_user(credentials = Depends(bearer_scheme), db = Depends(get_db))`, and then any route that needs an authenticated user just declares `current_user: User = Depends(get_current_user)` as a parameter. FastAPI notices the `Depends(...)`, runs the dependency function first (which itself has its own dependencies, resolved first), and only calls your route once every dependency has succeeded. If a dependency raises an `HTTPException`, your route body never runs at all and the client gets that error. This is how VR-Nexus enforces “every protected route requires a valid access token” and “some routes additionally require the admin role” without writing an authentication check by hand in eighty different functions; `app/api/deps.py` defines `get_current_user` and `require_role(...)` once, and every route that needs them just asks for them as a parameter.

Alembic migrations, and why you never hand-edit the schema. SQLAlchemy models describe what the database *should* look like in Python; Alembic is the tool that gets an actual live database to match that description, one small, versioned step at a time. Each migration is a Python file with an `upgrade()` and a `downgrade()` function, and Alembic keeps a table in the database recording which migrations have already been applied. If you connected to Postgres and manually ran `ALTER TABLE`, three problems follow immediately: nobody else on the team, and no other environment (staging, a teammate’s laptop, production), would automatically get that change; there would be no record of *why* the change happened or in what order relative to other changes; and Alembic’s own bookkeeping table would now disagree with reality, so the next real migration could easily fail or, worse, silently do the wrong thing. Treating the migration history as the single source of truth for schema changes, and never touching the schema any other way, is what keeps every copy of the database in the same shape.

Celery workers and tasks, and why long work does not run in the request handler. An HTTP request has an implicit contract: the client is waiting, and a reverse proxy, a load

balancer, or the browser itself will eventually give up if the server does not respond within some number of seconds. Extracting 150 requirements from a tender by calling an LLM once per chunk, or running OCR across 40 scanned pages, can easily take minutes. If that work happened inside the request handler, either the request would time out with the work still running (and the client would have no idea whether it succeeded), or you would need one web server thread tied up per slow upload, which does not scale. Celery solves this by splitting “describe the work” from “do the work”: your FastAPI route creates a database row describing the job and calls `some_task.delay(...)`, which serializes the task name and its arguments and drops them onto a queue (in this codebase, Redis). A separate long-running process, the Celery worker, watches that queue, picks up jobs, and executes the corresponding Python function, completely decoupled from any particular HTTP request. The web process and the worker process can even run on different machines. The request handler that queued the work returns to the client almost instantly, and the client finds out how the job is going by some other channel, here a WebSocket.

What Redis is doing here. Redis is an in-memory data store that this backend uses for three distinct jobs, and it is worth keeping them mentally separate even though it is the same running Redis instance. First, it is the Celery broker: the queue that task messages sit in between being created by a web request and being picked up by a worker. Second, it is a small key-value cache for progress state: whenever a Celery task advances a stage, it writes the latest progress payload to a Redis key with a TTL, and also publishes that same payload on a Redis Pub/Sub channel, so a WebSocket that connects late can read the key for “what’s the current state” and then subscribe to the channel for “what happens next.” Third, it holds short-lived, single-use tokens: WebSocket connection tickets (`app/services/ws_tickets.py`) and password-reset redemption markers (`app/services/password_reset_tokens.py`), both taking advantage of Redis’s atomic operations (`GETDEL`, `SET ... NX`) and automatic expiry (`EX`) to implement “usable exactly once, and only briefly” without any extra bookkeeping table.

WebSockets, and why progress uses one instead of polling. An ordinary HTTP request is one question and one answer; the connection then closes. A WebSocket is a single connection that both sides keep open and can send messages over in either direction, at any time, for as long as it stays connected. For a tender that takes two minutes to process, you could poll (have the browser ask “are you done yet?” every second) but that means either a noticeable delay between something actually changing on the server and the browser learning about it, or a lot of wasted requests that come back with “still working.” A WebSocket lets the server push a message the instant something changes, and the browser just reacts to it, using one connection instead of a hundred short-lived ones. The cost is that a WebSocket handshake cannot carry a normal Authorization header the way `fetch()` can, which is why this codebase authenticates its two progress sockets with a short-lived ticket passed as a query parameter instead (see `ws_tickets.py` in the file walkthrough below).

JWT access and refresh tokens. A JSON Web Token is a small, signed, self-contained piece of data: a payload (here, the user’s id, role, token type, issued time and expiry) plus a cryptographic signature that only the server, holding a secret key, could have produced. When a client presents a JWT, the server does not need to look anything up in a database to trust its contents; it just re-verifies the signature and checks the expiry. This backend issues two tokens on login: a short-lived access token (24 hours here, controlled by `ACCESS_TOKEN_EXPIRE_MINUTES`) that the client sends on every request, and a longer-lived refresh token (7 days) that the client stores and only uses to obtain a new access token once the old one expires, via `POST /api/auth/refresh`. Splitting the two means the token used on every single request is short-lived (so if it leaks, the exposure window is small) while the user does not have to log in with a password every day. A third, distinct token type, the password-reset token, is deliberately much shorter-lived (30 minutes) and, because it travels through email rather than staying in the browser, is additionally tracked in Redis so it can only ever be redeemed once, which a bare JWT cannot enforce on its own (see `app/core/security.py`

and `app/services/password_reset_tokens.py` below).

RAG and vector embeddings, and what pgvector adds to Postgres. An embedding is a way of turning a piece of text into a fixed-length list of numbers (a vector) such that texts with similar meaning end up as vectors that are numerically close together, typically measured with cosine similarity. This lets you do “search by meaning” instead of “search by exact keyword”: you embed a search query the same way you embedded your documents, then ask the database for the stored vectors closest to the query vector. `pgvector` is a Postgres extension that adds a native vector column type and fast nearest-neighbor operators (`<=>` for cosine distance, used throughout `app/services/library/search.py`) directly inside Postgres, so you do not need a separate specialized vector database; your embeddings live in the same tables, the same transactions, and the same backups as everything else. RAG, retrieval-augmented generation, is the pattern of using that search step as the first half of answering a question with an LLM: retrieve the most relevant passages first, then hand only those passages to the model and instruct it to answer using only what it was given, rather than letting it answer from its own general training. This backend’s `services/library/rag.py` is a compact, well-commented example of exactly that pattern, including citation tracking and an explicit refusal path when nothing relevant was found.

3. File-by-file walkthrough

The codebase is organized as one merged backend that grew out of two earlier prototypes: a “flat” prototype that had no real authentication or database, and a fuller tender-analysis system. Wherever you see a file whose entire content is a docstring plus a `raise ImportError(...)`, that is a **tombstone**: a leftover file from the merge that has been deliberately emptied so that importing it fails loudly and immediately, rather than silently running old, incompatible code. The project’s own convention (explained fully under `app/config.py` below, and worth internalizing as a pattern in its own right) is: when two modules of the same or similar name exist after a merge and only one is wired into the running app, do not just delete the loser and hope nobody references it by memory; replace its body with a `raise ImportError(...)` that names the real module. A stray `import app.config` then fails at the exact line that would otherwise have silently produced a half-configured app, instead of failing mysteriously three modules later, or not failing at all and just being wrong.

3.1 app/ root

`app/__init__.py` is empty. It exists only so Python treats `app/` as a package, allowing from `app.core.config` `import get_settings` style imports.

`app/config.py` is a tombstone superseded by `app/core/config.py`. It belonged to an earlier flat-layout prototype that had its own `Settings` class exposing a single `STORAGE_DIR`. The live configuration module splits storage into four separate roots and adds JWT, Celery and CORS settings the prototype never needed because its authentication was a hardcoded stub. The docstring is explicit that nothing in the running tree imports this file (verified by `grep` before it was emptied) and that it is safe to delete outright; it is kept only because whatever tooling performed the merge could not delete files. This file is the best place in the repo to read the tombstone pattern’s own reasoning in the author’s words, and it is worth reading once even though you should never import it.

`app/db.py` is a tombstone superseded by `app/core/database.py`. It defined its own SQLAlchemy engine, sessionmaker and declarative Base from the flat prototype. The docstring calls out the specific failure this tombstone exists to prevent: two separate Base classes in one process. If a second Base existed and some model accidentally registered against it, Alembic’s `autogenerate` (which walks `Base.metadata`) would not see those tables and would propose dropping them, and a `get_db` imported from this file would hand out

sessions bound to an entirely different connection pool than the rest of the app uses. This is a genuinely dangerous class of bug to have lying around importable, which is exactly why it raises instead of just sitting unused.

app/main.py is the live FastAPI entrypoint, the only one that should ever be run with uvicorn `app.main:app`. It builds the `FastAPI()` app, adds `CORSMiddleware` (needed because the frontend runs on a different origin, `localhost:5173`, than the API), conditionally mounts a static `/demo` directory if one exists on disk (resolved relative to the file itself rather than the process's working directory, so uvicorn still boots correctly no matter which folder you launch it from), and includes six routers: `auth`, `tenders`, `ws`, `library`, `library_ws`, and `health`. It also defines a plain GET `/health` endpoint that does a real `SELECT 1` against Postgres through the shared engine, so a liveness probe actually proves the database is reachable rather than just proving the Python process is alive. Every other route file plugs into the app only through this module; if you add a new router, this is where you register it.

app/models.py is a tombstone, but a different flavor from `config.py` and `db.py`: it is *unreachable by construction*, not merely unimported. Because `app/models/` exists as a package (a directory with an `__init__.py`), Python's import system always prefers the package over a same-named `.py` file sitting beside it, so `import app.models` can only ever resolve to `app/models/__init__.py`; this flat file is never executed at all, and its `raise` is documentation rather than an active guard. It still carries a full explanatory docstring pointing at the real package, which matters because a reader who opens the file in an editor has no way to tell from the filename alone that it never runs.

app/schemas.py is the same situation as `app/models.py`: shadowed and unreachable because `app/schemas/` is a package. Its docstring points to the three live schema modules (`auth`, `library`, `tender`).

3.2 app/api/

app/api/__init__.py and **app/api/routes/__init__.py** are both short docstring-only files that describe the layout rather than doing anything: HTTP concerns live under `app/api/`, `deps.py` holds the shared auth dependencies, and each router module under `app/api/routes/` owns its own `APIRouter(prefix=...)` so adding an endpoint never means touching `app/main.py`'s setup beyond one `include_router` call.

app/api/deps.py defines the two dependency functions almost every protected route in the app uses: `get_current_user` and `require_role(*roles)`. `get_current_user` takes the credentials extracted by FastAPI's HTTPBearer security scheme (which itself already returns 403 if the header is missing) and the DB session, decodes the JWT, checks that its type claim is specifically `access` (rejecting a caller who passes a refresh token where an access token belongs), looks up the user by the token's sub claim, and 401s if the token is invalid, expired, wrongly-typed, or the user no longer exists or has been deactivated. `require_role(*allowed_roles)` is a *dependency factory*, a function that returns a dependency function, which is the idiomatic FastAPI way to parameterize a check: `Depends(require_role(UserRole.ADMIN))` builds a fresh closure around the specific roles that route needs, layered on top of `get_current_user`, and 403s if the user's role is not in that set. The module's docstring makes an important conceptual point worth remembering as you move to other frameworks: FastAPI has no separate "middleware" concept for per-route authorization the way some other frameworks do; the idiomatic equivalent is exactly this, a dependency every protected route opts into.

app/api/library.py and **app/api/ws.py** are both tombstones from the flat prototype, superseded by `app/api/routes/library.py` and (jointly) `app/api/routes/library_ws.py` plus `app/api/routes/ws.py`. Both docstrings flag the same serious risk their existence as a tombstone protects against: the prototype versions were built against a no-op `DEV_PRINCIPAL`

stub instead of real authentication, so if either were ever accidentally registered on the live app (even by accident, even as a second, redundant registration), it would expose an entire unauthenticated copy of the Evidence Library API, or an unauthenticated WebSocket that anyone could connect to just by guessing an id. `ws.py`'s tombstone also explains why the live backend has *two* separate progress WebSocket routers rather than one shared one: the tender pipeline publishes progress on a Redis channel keyed by tender id with a 24-hour-TTL snapshot key, while the library indexing pipeline publishes on a differently-shaped channel keyed by index-job id and replays from the persisted `index_jobs` database row instead of a Redis snapshot. The two progress systems genuinely differ in payload shape and replay semantics, so merging them back into one router would be a real regression, not just tidying.

`app/api/routes/auth.py` implements registration, login, refresh, “who am I,” and the forgot/reset password flow. `register` hashes the password with `bcrypt` (via `app/core/security.py`) and deliberately has no role field on the request body at all, so nobody can self-elevate to admin over the API; the code comment is explicit that promoting a user to admin is meant to happen out of band. `login` implements the account lockout requirement: it checks `locked_until` before even checking the password, and on a wrong password increments `failed_login_attempts`, locking the account for `FAILED_LOGIN_LOCKOUT_MINUTES` once the count reaches `FAILED_LOGIN_LOCKOUT_THRESHOLD` (five, by default). It returns the exact same “Incorrect email or password” error whether the email does not exist or the password is wrong, which stops an attacker from using the login endpoint to discover which emails are registered. `refresh` decodes the refresh token, checks its type is specifically `refresh`, and mints a fresh access/refresh pair; note it does not invalidate the old refresh token, so this is a straight-forward stateless refresh rather than a rotating one. `forgot_password` and `reset_password` implement a full password-reset loop: a signed, 30-minute `password_reset` JWT is emailed as a link, and redeeming it goes through `app/services/password_reset_tokens.py`'s Redis-backed single-use check before the password is actually changed, specifically so a link sitting unread in an inbox for a week cannot be used twice even though the JWT itself would otherwise still verify. A successful reset also clears any active lockout, on the reasoning that a forgotten password is often the very reason an account got locked out in the first place.

`app/api/routes/health.py` exposes one endpoint, `GET /api/health/llm`, that round-trips a trivial prompt through the exact same Anthropic client configuration (`AsyncOpenAI` instance, base URL, model) that `services/extraction.py` uses for real tender extraction, and reports what happened. Its docstring explains the actual problem it solves: without it, an operator whose tender extraction fails has to guess whether the cause was a bad API key, a retired model name, a network block from the worker, or a spent quota, by re-running a full tender and reading worker logs. This endpoint answers that in a couple of seconds. It always returns HTTP 200, distinguishing “the probe ran and here is what it found” (ok: `true/false` in the body) from “the probe itself crashed” (which would be a 5xx); the design intent is that a caller checks one field in one response shape rather than two different response shapes depending on outcome.

`app/api/routes/library.py` is the live Evidence Library REST router. It splits upload and training into two separate calls on purpose: `POST /{category}/upload` only validates the file, computes a SHA-256 hash, checks for an exact or near duplicate against already-indexed documents (via `services/library/search.check_duplicate`), and stores the bytes plus a Document row in QUEUED state; nothing is parsed or embedded yet, because the duplicate check has to happen before any CPU is spent on a file that might just get rejected. `POST /train` is what actually enqueues Celery indexing jobs, either for a specific list of document ids or, if none are given, for everything still QUEUED; documents already INDEXED are skipped unless `force=true` is passed, which is what makes incremental training genuinely incremental rather than reprocessing the whole library every time one new asset is added. The rest of the router is largely `read/list/detail` endpoints, a `/search` endpoint that is the seam Stage 3 tender matching calls into directly, an `/ask` endpoint that calls into `ser-`

vices/library/rag.answer, and two file-serving endpoints (/documents/{id}/file for the original upload, /documents/{id}/images/{name} for extracted images) that both defer to `Path.resolve()` plus `is_relative_to(...)` checks against the storage root before serving anything, specifically to stop a crafted id or filename from being used to read arbitrary files off the server's disk. `_get_document` is written with an explicit comment that there is no per-user ownership check: the Evidence Library is a shared resource for the whole sales team by design, and `uploaded_by` exists for attribution, not access control.

app/api/routes/library_ws.py is the live library indexing progress WebSocket, at `/ws/library/{job_id}`. On connect it authenticates via a single-use ticket (see `services/ws_tickets.py`, described below), immediately replays the current `IndexJob` row from Postgres so a client that reloads mid-run does not sit on an empty progress bar, and then subscribes to that job's Redis Pub/Sub channel and forwards every frame it receives until a terminal stage (complete or failed) arrives. It periodically sends a `{"type": "ping", ...}` frame when nothing new has arrived within its poll timeout, both to stop an idle proxy from closing the socket during a long parse and to reliably notice a client that has gone away. The whole handler is wrapped in one `try` so that a client closing the tab the instant it connects (which can make even the very first `send_json` raise `WebSocketDisconnect`) is treated as a normal end of stream rather than an unhandled exception.

app/api/routes/tenders.py is the largest route module and covers the tender upload plus every read and review action that happens after the pipeline runs. `upload_tender` is deliberately thin: it validates the file, creates a `Tender` row in `UPLOADED` state, saves the bytes to disk, publishes an initial progress message, and calls `enqueue_pipeline(...)`, returning in milliseconds; everything from parsing onward runs in the Celery task described under `app/tasks/tender_pipeline.py`. `_get_tender` carries the same explicit “no owner check, this is a shared workspace” comment as the library router's `_get_document`. `review_match` implements the accept/reject/reassign action a human takes on one suggested piece of evidence, and re-fetches the row with its relationships eagerly loaded before serializing the response, since a plain `db.commit()` does not automatically refresh related objects the response schema needs. `cancel_tender` and `reprocess_tender` are the two routes that manipulate `run_generation`, the counter that guards against a stale worker overwriting a newer run's state; both routes bump the counter and the reasoning for exactly why is explained in depth under `app/tasks/tender_pipeline.py` below, since that is where the counter is actually enforced. `finalize_tender` recomputes the report and reassembles the output folder against whatever match decisions a human has made so far, then publishes a `FINALIZED` status, which is a terminal state for the progress socket. `report_tender_issue` is worth reading closely: its docstring states plainly that the actual failure email is composed and sent from the *user's own mailbox* via a frontend-opened `mailto: link`, and this endpoint only records that the hand-off happened (`support_requested_at`), so it does not itself send anything; that matters when you get to `services/support_email.py` below, because that service module implements SMTP sending for exactly this scenario but, as things stand in this codebase, is never actually called from any route. That is worth flagging as a concrete lesson: a fully-built, well-documented service module is not proof that it is wired into a live code path, and it is worth grepping for a function's call sites before assuming a docstring describes current behavior.

app/api/routes/ws.py is the live tender-progress WebSocket, `/ws/tenders/{tender_id}/progress`. It authenticates with a ticket exactly like the library socket does, then, on connect, calls `get_latest_progress(tender_id)` to read the Redis snapshot key and send it immediately (so a reconnecting client sees current state before any new event fires), and if that snapshot is already terminal (finalized or failed), closes the socket right there rather than waiting on a Pub/Sub message that will never come. Otherwise it subscribes to the tender's Redis channel and loops, using a short `get_message(timeout=1.0)` poll interleaved with a very short `receive_text(timeout=0.01)` check, so it notices a client disconnect promptly without blocking forever on either side. Unlike the library socket, this one talks to Redis using `redis.asyncio`

directly rather than going through a shared helper module, which is a small inconsistency worth noticing rather than imitating; it works, but it means the connection-management code is duplicated rather than shared between the two socket routers.

3.3 `app/celery_app.py`

This module constructs the single Celery application instance used by the whole backend, named "evidence_library" (a naming leftover from when only the library pipeline used Celery, now shared by the tender pipeline too), pointed at Redis as both broker and result backend, and explicitly lists its two task modules (`app.tasks.library_indexing` and `app.tasks.tender_pipeline`) via `include=...` rather than relying on autodiscovery. Its configuration choices are worth reading closely because each one encodes a real operational decision: `worker_prefetch_multiplier=1` stops one worker process from grabbing several queued jobs at once while a sibling worker sits idle, which matters because indexing and extraction are CPU- and I/O-heavy rather than trivially fast; `task_acks_late=True` means a job is only marked done after it actually finishes, so a worker that crashes mid-job does not silently lose that job; and `task_soft_time_limit/task_time_limit` give a long-running task a chance to record its own failure state before Celery kills it outright, rather than just vanishing. The final line, `celery_app.set_default()`, has a specific and slightly subtle reason documented in the file: Celery's `current_app` is thread-local, populated only on the thread that imported this module, and FastAPI runs synchronous route handlers on a worker thread pool, so without this call, a sync route trying to enqueue a task from a different thread could resolve `current_app` to a lazily-created default Celery app pointed at the wrong broker entirely (`amqp://localhost:5672`), producing a confusing "connection refused" even though the real Redis URL is configured correctly.

3.4 `app/core/`

`app/core/__init__.py` is a one-line docstring explaining that this package holds cross-cutting infrastructure with no dependency on `app/api` or `app/services`, so it can be imported from anywhere without creating an import cycle. That is a useful discipline to notice: `core/` is meant to sit at the bottom of the dependency graph.

`app/core/config.py` is the live Settings class, read once via `@lru_cache` `get_settings()` and used everywhere else in the app instead of scattering `os.getenv(...)` calls throughout the codebase. Reading it top to bottom is close to reading the whole backend's feature list: database and Redis URLs, JWT secret and lockout thresholds, four separate storage roots (root, tender, library, output), the embedding provider/model/dimension (with an explicit comment that the dimension is not a free parameter, since two vector(768) columns are already committed to it and changing it means a migration), one shared Anthropic API key powering three different features (tender extraction, library auto-tagging, and the grounded Ask), two different model names for cost tiering (explained fully under `services/extraction.py` below), and SMTP settings shared by two unrelated email features. The docstring on `ANTHROPIC_USER_AGENT` is worth reading on its own: an earlier version of this configuration deliberately overrode the outbound User-Agent header to impersonate Anthropic's own first-party CLI, in order to get past a third-party reseller's allowlist; this version removed that override because it now talks to Anthropic directly, and the comment states outright that impersonating another client was never something to keep, reseller workaround or not. That is a small but genuine lesson in how prototype code decisions do not always survive review, and why leaving a comment explaining *why* something was removed is as valuable as the removal itself.

`app/core/database.py` builds the single SQLAlchemy engine and SessionLocal sessionmaker, and defines `get_db()`, the FastAPI dependency essentially every database-backed route uses. `get_db` is a generator function: it creates a session, yields it (which is what lets FastAPI

hand that same session to your route function as `db: Session = Depends(get_db)`), and the code after the `yield`, inside a `finally`, runs once the request is finished, closing the session unconditionally whether the route succeeded or raised. `pool_pre_ping=True` is a small but practically important setting: it makes the connection pool test a connection before handing it out, which avoids a class of “server closed the connection unexpectedly” errors that show up after a database connection has sat idle for a while (a common annoyance on a laptop that just woke from sleep, or after a database restart).

app/core/security.py holds password hashing and JWT creation/decoding, kept in one file specifically because it has no dependency on the database or on FastAPI request objects, making it trivially unit-testable in isolation and reusable from anywhere. `hash_password/verify_password` wrap `passlib`’s `bcrypt` context. `_create_token` is the shared internal helper behind `create_access_token`, `create_refresh_token`, and `create_password_reset_token`, each of which just picks a different `TokenType` and expiry. `decode_token` deliberately only proves the signature and expiry are valid; it does not check which *kind* of token was presented, that check is left to each caller (`get_current_user` checks for `type == "access"`, the refresh route checks for `type == "refresh"`), which matters because otherwise a leaked long-lived refresh token could be used directly as an access token. `create_password_reset_token` additionally mints a random `jti` (JWT ID) claim precisely so the reset route can mark that one specific token as redeemed after use, something a bare JWT’s signature alone can never do since anyone holding an unexpired copy of a JWT can normally present it as many times as they like.

app/core/ws_manager.py defines `TenderProgressConnectionManager`, an in-process registry mapping a tender id to the set of currently-open WebSocket connections watching it, with a `broadcast()` method that pushes a JSON payload to every socket watching one tender and quietly drops any socket that has gone stale. Its own docstring is candid that this design only works because it assumes a single Python process holding both the WebSocket connections and the code that produces updates, and explicitly warns that once the real pipeline runs in a separate Celery worker process, `broadcast()` would need to go through Redis Pub/Sub instead, since two separate OS processes cannot share one Python dictionary. That is exactly what happened: the live tender pipeline does run in a Celery worker, and the live progress route (`app/api/routes/ws.py`) was rewritten to talk to Redis directly rather than to this manager. A `grep` across the whole codebase confirms `ws_manager` is not imported anywhere outside this file itself; the module is genuinely dead code in the running app, but unlike the files under `app/config.py` and its siblings, it was never converted into a tombstone that raises on import, it is simply unreferenced. That is a useful thing to notice as a student: dead code does not always announce itself, and “`grep` for who imports this” is a habit worth having whenever you are not sure whether a module you are reading actually runs.

3.5 app/models/

app/models/__init__.py imports every model class so that importing the package as a whole registers all of them on `Base.metadata`. Its docstring states the real reason this matters: Alembic’s `env.py` imports this module specifically so that `alembic revision --autogenerate` can see every table when comparing the current models against the live database schema. If a model file existed but was never imported anywhere, Alembic’s `autogenerate` would simply not see it and could propose dropping tables that model actually needs, purely because nothing pulled that file into memory before the comparison ran. This is the concrete, practical reason “import everything in one place” is a pattern worth following rather than an arbitrary style choice.

app/models/base.py defines the shared `Base` declarative class every model inherits from, plus two mixins, `UUIDPKMixin` (a client-side-generated UUID primary key, chosen so the app does not depend on a Postgres extension like `pgcrypto` being enabled for server-side UUID genera-

tion) and TimestampMixin (timezone-aware created_at/updated_at, both defaulted in application code rather than by a database-side trigger, so the same value is guaranteed no matter which code path writes the row). Almost every model in the package inherits both mixins, which is why you will not see created_at or id redeclared table by table.

app/models/enums.py centralizes every enum shared across models, and each one is written to Postgres as a native ENUM type via SQLAlchemy's Enum(..., name=...), rather than a plain string column, so the database itself rejects a value outside the allowed set at the storage layer, not just in application code. Reading this file top to bottom is a fast way to understand the shape of the whole domain: UserRole (only admin/user, with a comment noting the original spec's third role was folded into user), DocumentCategory and DocumentFileType for the library, DocumentTrainingStatus and the closely related but deliberately distinct JobStage (the former is "what state is this document in right now," the latter is "how far did this specific training run get," and re-training an already-indexed document starts a fresh job at queued while the document itself correctly stays indexed until the new run finishes and replaces its chunks), the two lookup tables JOB_STAGE_LABELS and JOB_STAGE_TO_DOCUMENT_STATUS that keep the job-stage-to-document-status mapping in exactly one place so the two can never silently drift apart, TenderStatus (the eight pipeline stages plus the two terminal states), EvaluationImpact, RequirementStatus, and MatchType/MatchReviewStatus for the evidence-matching review workflow. The comment on MatchType is worth noting: it states the exact confidence thresholds (0.85 for auto, 0.60 to 0.84 for suggested) and explicitly instructs that these must be kept in sync with the equivalent constants in app/tasks/tender_pipeline.py, an example of a business rule that has to live in two places (an enum's documentation and a task's actual threshold constants) because the enum itself cannot enforce a numeric range.

app/models/user.py defines the User table: name, email (unique, indexed), a bcrypt password hash, role, an is_active flag, the two columns that implement account lockout (failed_login_attempts, locked_until), and last_login_at. Its relationships (uploaded_documents, uploaded_tenders, audit_logs) are declared with explicit foreign_keys= arguments because more than one foreign key in this schema can point at users.id (for example, both uploaded_by and reviewed_by on different tables), and SQLAlchemy needs to be told unambiguously which column a given relationship should join on whenever more than one plausible column exists.

app/models/document.py defines Document, the single table backing all three Evidence Library categories (case study, methodology, company document) rather than three separate tables, on the reasoning that the categories differ only in how a file is parsed, not in what metadata it needs to carry. It carries the LIB-UI-03 metadata fields (client, sector, service line, geography, keywords), a unique, indexed file_hash used for the exact-duplicate check, the LIB-UI-05 training status, and the Section 6 additions layered on top (doc_type, auto_tagged_fields, page_count, chunk_count, indexed_at). DocumentImage is a small child table recording where an extracted image was written on disk, by path only; images are never stored as pixel data inside a row.

app/models/chunk.py defines Chunk, one embedded passage of a library document, with a UniqueConstraint on (document_id, chunk_index) and an embedding column typed Vector(EMBEDDING_DIM) from the pgvector SQLAlchemy integration. It denormalizes category from the parent Document directly onto the chunk row specifically so a category filter can be applied to a vector search without first joining back to documents, which matters for query performance once the library has many chunks.

app/models/index_job.py defines IndexJob, one row per Train/Memorize run, existing specifically so training progress survives a page reload; it is the persisted counterpart to the live Redis Pub/Sub frames the library WebSocket streams, and its own stage/progress/message columns are exactly what library_ws.py's snapshot replay reads on connect.

app/models/audit_log.py defines a deliberately generic AuditLog table (action, re-

source_type, resource_id, a free-form details JSON blob) so that one table can log logins, tender runs, document training and user-management actions without a separate table per action type; user_id is nullable because some events (a failed login attempt against an unknown email, or a system-level error) have no authenticated user to attach the entry to.

app/models/tender.py defines Tender, the largest single model in the app, tracking one uploaded document through all eight pipeline stages plus best-effort metadata (reference id, issuing authority, sector, location, value, deadline) filled in by a single LLM pass and correctable by the user afterward via PATCH. Its comment block on the failure-record columns (failed_stage, error_detail, failed_at) explains a real design problem worth understanding: progress_message alone is not enough to build an error log, because every pipeline stage overwrites it, so by the time a run has failed there is no record of *which* stage actually died or what the underlying exception was, once a retry has started overwriting the same column again. The three dedicated failure columns exist purely to survive that overwrite, and all three are explicitly cleared on a successful reprocess so the row always describes only the current attempt. The run_generation and celery_task_id columns are the mechanism behind cancel/reprocess safety and are explained in depth under app/tasks/tender_pipeline.py, since that is where they are actually enforced rather than merely stored.

app/models/requirement.py defines Requirement (one row per extracted requirement, with both a normalized column and a verbatim _raw column for several fields, because real tenders write compound or non-boolean values like “No/Advisory” or “Financial / Pass-Fail” that a clean enum or boolean cannot hold, and the output tracker has to reproduce the tender’s own wording exactly) and RequirementEvidenceMatch, a separate child table rather than a single foreign key on Requirement, specifically because a requirement can have more than one candidate evidence document (auto-matched, several suggested candidates, or a “missing, here’s the closest we found” candidate) each with its own confidence score and its own independent accept/reject/reassign review history.

app/models/tender_chunk.py defines TenderChunk, and its docstring is a clear worked example of why two similarly-named tables should sometimes stay separate rather than be merged: Chunk (library) and TenderChunk (tender) are both “a piece of a document with an embedding,” but they have different owners, different chunk sizes and overlap, and different downstream consumers, so conflating them would make both harder to reason about even though the underlying idea is the same.

3.6 app/schemas/

app/schemas/__init__.py is a short docstring repeating the request/response versus database boundary explained in the core concepts section above.

app/schemas/auth.py defines the request and response shapes for the auth endpoints: RegisterRequest (with password: str = Field(min_length=8, max_length=72), the 72-character maximum being bcrypt’s own hard input limit, not an arbitrary choice), LoginRequest, RefreshRequest, the forgot/reset password request pair, UserOut (configured with model_config = {"from_attributes": True}, which is what lets Pydantic build this schema directly from a SQLAlchemy User object’s attributes rather than requiring a plain dict), and TokenResponse, which nests a UserOut inside itself so a successful login returns both tokens and the user’s own profile in one response.

app/schemas/library.py is the largest schema module and defines everything the library router returns: MetadataInput for uploader-supplied fields, ChunkOut/DocumentOut/DocumentDetailOut for read responses, UploadResponse plus the DuplicateMatch/DuplicateCheckResult pair for LIB-UI-06’s duplicate warning, TrainRequest/TrainResponse/JobOut for the training flow, WsTicketOut for the library progress ticket, ProgressEvent describing exactly what crosses the WebSocket, SearchHit/SearchResponse for plain retrieval, and AskRequest/AnswerOut for

the grounded-answer endpoint. A small but genuinely useful pattern shows up here repeatedly: several fields use `field_validator(..., mode="before")` helpers like `_blank_if_none` and `_empty_list_if_none` to normalize a NULL database value into an empty string or empty list before validation runs, specifically because the underlying columns are nullable (rows can have been written by a different code path than the one that always writes ""), and the API contract promises clients one consistent shape (never null where a string or list is expected) regardless of which code path produced the row.

app/schemas/tender.py defines the tender-side schemas, and its module docstring explains a small, easy-to-miss decision worth remembering for your own projects: money and marks columns are Numeric in the database (for exact decimal precision) but are typed as plain float in these Pydantic schemas, because Pydantic serializes a Decimal to a JSON *string* in its default JSON mode, which would force the frontend to parse every currency and marks value back into a number itself; keeping the wire format as a JSON number and letting the database column remain the actual source of truth for precision is a deliberate tradeoff, not an oversight. TenderReportOut's field comments are worth reading closely for how they define "coverage": a requirement only counts toward requirements_with_evidence/requirements_without_evidence if evidence_required is true in the first place, because a narrative clause the tender never asked the bidder to prove would otherwise get mislabeled as "missing evidence" purely because nobody ever tried to match it.

3.7 app/services/ (root)

This is the directory where the "old prototype versus live module" split is most extensive, and it repays reading carefully because it demonstrates the tombstone pattern at scale rather than in one or two isolated files.

app/services/__init__.py states the organizing principle directly: two families of service modules live under `app/services/`, the tender pipeline (chunking, PDF extraction, extraction, pipeline stage labels, progress, tender storage, all directly under `app/services/`) and the Evidence Library (entirely under the `app/services/library/` subpackage), and they are not merged because chunking and progress exist under both names with genuinely different behavior, which is exactly why the library's copies are namespaced under `library/` instead of living flat beside the tender versions.

app/services/chunking.py is live, not a tombstone, and it is easy to mistake for one given how many of its neighbors are tombstones. It implements section-aware chunking for tender documents: detecting section boundaries by regex (SPN, Instructions to Proposers, PDS, Evaluation Criteria, Section VII, Annexes, falling back to a generic "General/Front Matter" label), grouping consecutive pages by detected section, then splitting each section into roughly 2000-token windows with roughly 200 tokens of sliding overlap between consecutive chunks. What makes this module worth reading closely is the comparison the docstring draws against the prototype it replaced: the prototype tracked only a page *range for the whole section*, then blindly token-split the concatenated text, so any chunk after the first one in a multi-chunk section had no real page attribution at all; this version tokenizes per page and keeps a running token-offset-to-page map (using `bisect` for an efficient lookup) so every individual chunk gets a true `page_start/page_end`. It also had genuinely no overlap at all in the prototype despite the spec calling for it, meaning a requirement that happened to fall across a chunk boundary could be silently lost; this version actually slides the window. `guard_against_whole_document_ingestion` is a small but important safety check: it raises if any chunk exceeds the token budget or if chunking produced zero chunks for a non-empty tender, both of which would otherwise let something oversized or silently empty flow into the expensive LLM extraction stage undetected.

app/services/embeddings.py, **app/services/metadata.py**, **app/services/rag.py**, **app/services/search.py**,

app/services/storage.py, and **app/services/llm.py** are all tombstones, each superseded by its like-named counterpart under **app/services/library/**. Their docstrings share a consistent shape worth noticing as a pattern: each one names the exact live replacement, explains the concrete technical reason the old copy cannot simply be revived (it was built against the tombstoned flat **app.config/app.db/app.models**, or against database columns a later migration added that the old model definition does not have, or, in **llm.py**'s case, against a User-Agent impersonation setting that no longer exists on the live **Settings** class at all), and states plainly that the file is safe to delete and is only kept because the merge tooling could not delete files. **storage.py**'s docstring is worth a specific mention: it explains that the flat prototype resolved everything under one **STORAGE_DIR** setting, and because the live **Settings** class is configured with **extra="ignore"**, a stray **STORAGE_DIR** entry left in a **.env** file would not raise an error, it would simply be silently swallowed and paths would quietly fall back to their defaults; that silent-failure risk is exactly why this module was tombstoned outright rather than just left unused.

app/services/parsers/ (whole subpackage), **__init__.py**, **base.py**, **case_study.py**, **company.py**, **extract.py**, **methodology.py**, **registry.py**, is a tombstone package superseded wholesale by **app/services/library/parsers/**. The package **__init__.py** raises on its own, which is what makes every submodule inside it unreachable even before Python gets to executing them individually (each submodule still carries its own explanatory raise as “a second line of defence for anyone who copies the file elsewhere,” in the codebase’s own words, which is a reasonable defensive habit: a tombstone that only guards at the package level can be defeated by copying one file out of the package). The package docstring explains why the live parsers live under **library/** specifically rather than being merged flat: parsing is one of the places the tender side and the library side genuinely differ in kind, not just in tuning, since the library parsers are section-oriented and produce library **Chunk** rows while the tender side parses page-oriented PDFs through **pdf_extraction.py** and **chunking.py** into a different dataclass entirely.

app/services/password_reset_tokens.py is live and implements single-use redemption tracking for the password-reset JWT described earlier. Its **redeem(jti, ttl_seconds)** function is a small, worked example of a genuinely important concurrency pattern: it uses **SET ... NX** (set-if-not-exists) against Redis, which performs the “is this already used, and if not, mark it used” check as one atomic server-side operation, returning **True** only the first time a given **jti** is redeemed and **False** on every call after, including a second request racing the first by microseconds. A naive “read the key, check if it exists, then write it” implementation would have a race window between the read and the write where two near-simultaneous requests could both see “not yet redeemed” and both succeed; **SET NX** closes that window entirely by doing both steps as one indivisible operation.

app/services/pdf_extraction.py is live and extracts per-page text plus tables from a tender PDF using **PyMuPDF**, falling back to **Tesseract** OCR on any page whose extracted text is suspiciously short (under **OCR_FALLBACK_CHAR_THRESHOLD**, on the reasoning that a page with real text rendered as an image is the signature of a scan). It is built on the same core approach as an earlier prototype, but the docstring flags one specific, easy-to-miss bug the prototype had: it hardcoded a Windows path to the **Tesseract** binary at *import time*, which is global to the **pytesseract** module, meaning it silently redirected the Evidence Library’s own, separate OCR path as an unrelated side effect of merely importing this module, and it broke entirely inside Docker, where the binary lives somewhere else. This version makes the **Tesseract** path opt-in via an environment variable, verified to actually exist on disk before being applied, and falls back to resolving **tesseract** from **PATH** otherwise, which is a small but real lesson about avoiding module-level side effects that reach outside the module doing them.

app/services/tender_storage.py validates (must be a **.pdf**, under 100MB, non-empty) and saves an uploaded tender to disk under **TENDER_STORAGE_DIR/{tender_id}/{filename}**, keyed by the tender’s own **UUID** so two uploads that happen to share a filename never collide. Its

docstring explains why it is named `tender_storage.py` rather than the more generic `storage.py`: keeping tender uploads and library uploads in separate modules, each targeting its own storage root, avoids exactly the kind of accidental grab-bag module two unrelated features both end up depending on, which is the same discipline `app/services/storage.py`'s tombstone explains from the other direction.

`app/services/extraction.py` is the LLM-based requirement extraction engine and one of the most heavily-commented files in the repo; it is worth reading in full once you understand the rest of the pipeline. `run_extraction` fans out one LLM call per `TenderChunk`, bounded by an `asyncio.Semaphore(EXTRACTION_CONCURRENCY)` so a large tender's 150-plus chunks run several at a time rather than either one at a time (slow) or all at once (likely to hit a provider rate limit), consumed via `asyncio.as_completed` specifically so the progress bar advances as each chunk finishes rather than jumping all at once at the end. Every three completed chunks (and always on the last one) it re-reads the Tender row from the database and, if a user has pressed Stop in the meantime (which sets the row's status to FAILED from a completely different request), cancels every still-running task and raises `TenderCancelled`; this is the cooperative half of the cancellation mechanism, checked from inside a loop that is otherwise just waiting on network calls it cannot interrupt any other way. `_hash_requirement` deduplicates extracted requirements by a hash of their clause reference plus description, since chunk overlap means the same requirement can genuinely be extracted twice from two overlapping chunks. The failure logic at the end of `run_extraction` is deliberately strict and ordered: if more than 30% of chunks failed after three retries each, or if zero requirements were extracted at all, the run is raised as a hard `ExtractionFailed` rather than allowed to continue to matching and reporting with a silently incomplete requirement set, because, in the module's own words, a tender extracted with 40 of 400 requirements and presented as a finished result with nothing anywhere saying so is a worse outcome than a run that fails loudly and tells the operator exactly what to check. `EXTRACTION_MODEL` is read from `ANTHROPIC_EXTRACTION_MODEL` rather than `ANTHROPIC_MODEL`, the same cost-tiering decision explained under `services/library/llm.py` below: this is high-volume, mechanical, "copy the text verbatim into these fields" work run over a hundred times per tender, which a cheap fast model does as reliably as an expensive one.

`app/services/pipeline_stages.py` is live and defines the ordered list of the eight tender pipeline stages plus a human-readable label for each, so the frontend does not have to turn `ASSEMBLING_FOLDER` into "Assembling Output Folder" itself. Its own docstring flags a real, deliberately-unfixed inconsistency: `app/services/progress.py` carries a second, near-identical `PIPELINE_STAGES/STAGE_LABELS` pair with slightly different label text ("Parsing" versus "Parsing Document," for instance), and `progress.py`'s copy is the one that actually reaches the client, since `publish_progress()` builds its payload from its own dictionary rather than this module's. This module's labels are only used by code that calls `stage_label()` directly. The docstring explicitly calls this duplication out rather than silently fixing it, on the reasoning that consolidating the two would change strings the frontend may already depend on, so it belongs in a deliberate, coordinated change rather than a quiet cleanup. That is a good habit to notice: not every piece of duplication found while reading code should be "fixed" reflexively, especially when the fix would change an external contract.

`app/services/progress.py` is the live Redis-backed progress publisher for the tender pipeline, described in the core concepts section above: for each tender it maintains a String key holding the latest progress payload as JSON (read on connect or reconnect, for TRK-04 resume behavior) and a Pub/Sub channel of the same logical name that is published to on every update (read live by an open WebSocket). `publish_progress` computes a percentage from the tender's position in `PIPELINE_STAGES` when no explicit percent is given, and the module has no FastAPI import at all, which is what lets the exact same function be called equally validly from an ordinary request handler or, as things stand today, from inside a Celery worker process, without either caller needing to know anything about how a WebSocket eventually delivers

that update to a browser.

app/services/account_email.py sends the password-reset email over plain SMTP (smtplib, STARTTLS, no third-party email service), running the actual blocking send inside `asyncio.to_thread` because smtplib itself has no async API and the calling endpoint is async. Its docstring draws a careful distinction against its sibling `support_email.py`: both share the same SMTP transport mechanics, but `support_email.py` is deliberately never wired to a user-supplied recipient (so it can never be turned into an open relay by anything a tender or a form contains), while this module's entire job is to email a user-supplied address, so that guarantee does not and should not apply here. The two modules duplicate a small amount of SMTP boilerplate rather than sharing a helper specifically to keep that trust boundary visible and unambiguous, rather than blurred behind a shared "internal" function used by both a safe and an unsafe caller.

app/services/support_email.py composes and sends a tender's full failure detail (stage, reason, exception, stack tail, tender id) to a single configured support address, again over plain SMTP. Its docstring explains the actual product reasoning: VR-Nexus is used by non-technical procurement staff, and a raw Python traceback on screen helps nobody who cannot read it while alarming anyone who can, so the "Contact technical support" flow exists to route that traceback to people who can act on it while leaving the user with a calm, non-technical message. As already noted under `routes/tenders.py` above, this is the one place in the codebase where a fully-implemented, carefully-documented, non-tombstoned service module is nonetheless never actually called; the frontend implements the "contact support" hand-off through a `mailto:` link opened in the user's own mail client instead, and the backend route only persists that the hand-off occurred. `send_failure_report` remains here, correctly written and ready to use, but currently orphaned.

app/services/ws_tickets.py is the module the task description calls out as a model for tone and depth, and it earns that reputation. It solves one precise problem: a browser's native Web-Socket constructor cannot set custom headers, so it cannot carry an `Authorization: Bearer <token>` the way a normal `fetch()` call can, and the naive fallback (put the real access token in the connection URL as a query parameter) means a long-lived, 24-hour bearer credential ends up sitting in browser history, in every proxy access log the connection passes through, and potentially in a `Referer` header, for as long as that token remains valid. The fix is the standard one for exactly this shape of problem: `issue_ticket(user_id, scope, resource_id)` is called from an ordinary, already-authenticated HTTP endpoint (one that *can* carry a real `Authorization` header, and whose access logs matter far less than a URL that lands in browser history), mints a random 32-byte token, and stores it in Redis with a 60-second TTL, scoped to a specific (`scope, resource_id`) pair, for example ("tender", "<tender-id>"), so a ticket minted for one tender's progress socket cannot be pointed at a different tender's. `redeem_ticket` is where the file's most important technical detail lives: it calls Redis's `GETDEL` rather than a plain `GET` followed by a separate `DELETE`. The reasoning given is precise and worth internalizing as a pattern: a WebSocket handshake is exactly the kind of operation a client can retry, double-fire, or race against itself (a React effect re-running, a reconnect attempt racing the original connection), and a plain `GET` would let two near-simultaneous redemption attempts both see the ticket as still valid before either one deletes it. `GETDEL` reads and deletes the key as one atomic server-side step, so only the first caller to reach Redis can ever succeed, and the second sees exactly nothing, as if the ticket had never existed at all, not a race, not a rare bug, structurally impossible by construction.

3.8 app/services/library/

app/services/library/__init__.py repeats, briefly, why this exists as a subpackage rather than flat modules alongside the tender-side services: the tender side already owns modules named `chunking` and `progress`, so the library's own versions are deliberately namespaced

here instead.

app/services/library/chunking.py splits a parsed section's text into roughly 800-token windows with 100 tokens of overlap, always staying within one section (never spanning, say, a case study's "Challenge" section into "Solution"), so that a chunk's section/phase/page labels stay semantically meaningful and every embedded vector represents one coherent idea rather than a boundary-straddling mash of two.

app/services/library/embeddings.py defines the `EmbeddingProvider` abstract base class and the one concrete implementation currently registered, `FastEmbedProvider`, which runs the BAAI/bge-base-en-v1.5 model locally via the `fastembed` library (ONNX runtime, CPU-only, no external API call at all). The docstring explains a real architectural decision made partway through the project: the original implementation plan called for a hosted embedding API, but the team deliberately switched to a fully local model specifically to avoid adding a *second* API-key dependency on top of the one Anthropic key that already powers extraction, tagging and Ask, and because Anthropic itself does not serve an embeddings endpoint at all, so there was no way to keep embeddings on the same provider even if a hosted model had still been wanted. The model is loaded lazily, behind a `threading.Lock`, on first use rather than at import time, because Celery forks worker processes, and eager loading at import time would pay the (non-trivial) model-loading cost in every process including the FastAPI web process itself, which never computes an embedding on its own. `embed_documents` explicitly checks that the returned vector width matches `EMBEDDING_DIM` and raises immediately with an actionable message if it does not, rather than letting a dimension mismatch surface later as an opaque `pgvector` insert failure.

app/services/library/library_progress.py is the library-side twin of the tender side's `progress.py`, publishing to a differently-namespaced Redis channel (`library:progress:{job_id}` versus `tender:{id}:progress`) and never raising on a publish failure, on the explicit reasoning that a dropped progress frame should never be allowed to fail the indexing job that was trying to emit it.

app/services/library/llm.py wraps a single `openai.OpenAI` client instance pointed at Anthropic's OpenAI-compatible endpoint, used for parsing fallback and metadata auto-tagging. `complete()` returns an empty string on any failure rather than raising, and its docstring is explicit that this is deliberate: every call site in the codebase is written to tolerate a missing or failed LLM response and fall back to heuristics, so with no `ANTHROPIC_API_KEY` configured at all, the entire library pipeline still runs end to end, just without machine-assisted tagging. Its own docstring draws the same contrast as the tombstoned `services/llm.py` did, but from the winning side: the earlier version impersonated Anthropic's own CLI via a `User-Agent` override to satisfy a third-party reseller's allowlist, and this version does not, because talking to Anthropic directly means there is no reseller allowlist left to satisfy.

app/services/library/metadata.py implements LIB-IDX-06 auto-tagging with a strict, explicitly documented precedence order: whatever the uploader typed into the form always wins outright and is never touched; keyword-and-regex heuristics (sector keyword maps, service-line keyword maps, a geography gazetteer, a client-label regex) fill in whatever the user left blank; and the LLM is consulted only for fields still blank after both of those, using the cheap extraction-tier model rather than the reasoning model, on the same "mechanical field-filling" cost logic used everywhere else in the codebase. `auto_tagged_fields` records exactly which fields were machine-filled so a human reviewer can visually distinguish a typed value from a guessed one later in the UI.

app/services/library/rag.py is the generation half of the RAG pattern described in the core concepts section, and its own docstring states the design rule directly: the model is never treated as a source of facts, it only ever sees the chunks retrieval already found, is instructed to answer using only those chunks, and must produce an exact, checked-for sentinel string when the context does not answer the question. `_format_context` numbers each

retrieved chunk as [1], [2], and so on and truncates on a whole-chunk boundary rather than mid-sentence once a hard character cap is hit, specifically because cutting a claim off mid-sentence can strand it from the qualifier that limits its meaning. `_cited_indexes` then parses the model's own answer text for exactly those bracketed numbers and marks each source hit as cited or not; a source that was retrieved but never cited is still returned to the caller rather than filtered out, because knowing what the model considered and chose not to use is itself useful information for a reviewer. `answer()`'s grounded flag is computed strictly: true only when the model produced at least one citation and did not decline, which means an answer that came back with prose but with nothing cited is correctly reported as ungrounded, treating an uncited answer as exactly the failure mode this whole design exists to catch, rather than quietly hiding it behind a confident-looking response.

app/services/library/search.py performs cosine-distance vector search directly against Postgres via `pgvector`'s `<=>` operator (`Chunk.embedding.cosine_distance(vector)`), converting distance to similarity as `1 - distance`. Its docstring states a deliberate architectural choice: there is no in-memory index and no result cache anywhere in this module, every query goes straight to the live database, which is exactly what makes "a document indexed a moment ago is immediately searchable" true by construction rather than something that needs an invalidation step to keep correct. `find_near_duplicates` over-fetches (`limit * 10`) before collapsing to one best match per document, since several chunks belonging to the same document will naturally rank adjacently in a similarity search and would otherwise be counted as several separate candidate matches instead of one.

app/services/library/storage.py validates uploads by extension (deliberately not trusting the client-reported MIME type, since browsers are inconsistent about it for Office formats, and a genuinely malformed file will fail loudly during parsing anyway) and writes files to disk under a UUID-derived name rather than the client-supplied filename, which is a specific and important security detail: `save_upload`'s docstring notes plainly that because the original filename is only ever used for its file extension, a crafted filename like `../../etc/passwd` cannot be used to escape the storage directory, since nothing about the client's filename text ever becomes part of the destination path beyond its suffix.

app/services/library/parsers/ is the live parser package, distinct from and superseding the tombstoned **app/services/parsers/** sibling described above. **__init__.py** is a one-line docstring. **base.py** defines the shared vocabulary every parser speaks: `Page` (one page/paragraph-block/slide of format-agnostic extracted content), `RawDoc` (the format-agnostic extraction result, with a `page_for_offset()` helper that maps a character offset in the joined full text back to the page it came from, which is what lets a downstream section parser, working purely on joined text, still attach an accurate page number to each section it finds), `Section` (a logical, chunk-ready unit of a document), `ParseResult`, and a small module-level `_REGISTRY` dict populated by each category parser calling `register(category, parser)` at import time. **registry.py** exists purely to trigger that registration by importing all three category parser modules, and its own docstring explains why it is a separate module rather than folded into **__init__.py**: importing `base.py` alone (for its dataclasses, say, from a test) should not have to drag in `PyMuPDF`, `python-docx` and `python-pptx` as a side effect, and putting the registration imports in **__init__.py** would force exactly that. **extract.py** is the format-specific extraction layer (PDF via `PyMuPDF`, DOCX via `python-docx`, PPTX via `python-pptx`, plus a bare-image path that OCRs the whole file as a single page), and its docstring is explicit about a rule enforced throughout: embedded images are written to disk and only their *paths* ever travel onward into chunk content, never raw pixel bytes or a base64 encoding, on the reasoning that a vector of pixels is not comparable to a vector of text, and storing image data inside a text column would only poison retrieval quality for no benefit. It also exposes `sample_text()`, a deliberately cheap, request-path-safe text sample (a handful of pages, no images, no OCR) used only for the upload-time duplicate check, kept separate from full extraction specifically because full extraction is far too slow to run synchronously

inside an HTTP request handler.

app/services/library/parsers/case_study.py tries to resolve a case study’s text into the six canonical sections (Client, Sector, Scope, Challenge, Solution, Results) by matching a broad synonym table against detected heading-shaped lines (a synonym lookup built once at import time, sorted longest-first so a more specific phrase like “project scope” is matched before a bare “scope”). If fewer than three of the six sections can be found this way, it falls back to asking the LLM (on the cheap extraction-tier model) for a section map instead, and only if that also fails does it index the document as one single undifferentiated blob, which the module’s own docstring frames correctly as the last resort rather than the second choice. When the LLM path is used, the verbatim source text is kept alongside the model’s own section summaries as an extra “Full Text” section, specifically so retrieval can still quote the source document directly rather than being limited to the model’s paraphrase of it.

app/services/library/parsers/company.py handles certificates, registrations, tax filings and similar short single-purpose documents, and its docstring explains why it deliberately does not split these into sections at all: splitting an ISO certificate into “sections” would produce fragments like “Scope of certification” divorced from the certificate number that gives that fragment any meaning, which is worse for retrieval than simply keeping the whole short document as one unit. What this parser does instead is classify the document type using a weighted keyword-scoring table, falling back to the LLM only when the keyword score is too low to trust, and extract structured identifiers (certificate number, NTN, issue and expiry dates, issuing authority) via a set of regexes, landing all of that on the document row rather than on any individual chunk.

app/services/library/parsers/methodology.py splits a methodology document by detected phase markers (“Phase 2”, “Stage III”, “Step 4”, numbered outline headings, or markdown headings promoted from Word/PowerPoint styles by `extract.py`), matched in that priority order, and falls back to asking the LLM for an ordered phase list only when no markers are visible at all. Its docstring explains why phase ordering specifically matters here in a way it would not for a case study: methodology documents are inherently sequential, and when the library is later asked something like “what is your approach to data migration,” the phase label riding along on the retrieved chunk tells a proposal writer where in the overall plan that passage belongs, which a plain similarity score alone cannot convey.

3.9 app/tasks/

app/tasks/__init__.py is a short docstring stating plainly that `library_indexing.py` is the live task module and `indexing.py` beside it is a tombstone, and flags a subtlety worth remembering: the tasks registered inside `library_indexing.py` deliberately keep the *old* `app.tasks.indexing.*` registered names rather than being renamed to match their new file location.

app/tasks/indexing.py is a tombstone, but its docstring is worth reading closely because it explains something a simpler tombstone would not need to: a Celery task’s registered name (the string passed to `@celery_app.task(name=...)`) is the routing key carried inside every message a worker ever receives, entirely independent of which Python file that task’s code happens to live in today. If the tasks had been renamed to match their new module path when they moved, any message already sitting in the Redis queue at deploy time (queued under the old name) would arrive at a worker that no longer has a task registered under that name, and would simply be rejected. So the *module* moved to `library_indexing.py`, but the *registered names* stayed exactly as they were, and this tombstone exists specifically to stop anyone from accidentally re-registering a second, competing set of tasks under those same names by importing this old file, since Celery’s rule for a name collision is that the last registration to run wins, meaning which actual implementation processes a given job would end up depending on unpredictable import order.

app/tasks/library_indexing.py is the live Celery task implementing the whole LIB-UI-04/05 indexing pipeline, one task per document, which is what makes incremental training actually incremental: nothing in this file ever reads or rewrites another document's rows, so training one new asset never touches the rest of the library. `_advance` is the shared helper every stage calls: it writes the new stage, progress percentage and message onto the persisted `IndexJob` row, optionally mirrors that stage onto the parent Document's `training_status` column (routed through the `JOB_STAGE_TO_DOCUMENT_STATUS` lookup table from `models/enums.py` so the two columns can never be set to disagreeing values), commits, and *then* publishes the same update over Redis. Doing both, in that order, is deliberate: the WebSocket carries live updates for whoever is watching right now, but a client that connects late or reloads mid-run reads the persisted row instead of simply missing whichever stage happened while nobody was looking. `_run_pipeline` walks Parsing (calling into `parsers.extract`, then the category-specific parser via `registry.parse`), Tagging (calling `metadata.extract`, carefully distinguishing what the uploader typed from what the parser itself already inferred from what the metadata module is allowed to overwrite), and Embedding (chunking the parsed sections, deleting this document's own previously-indexed chunks and rebuilding them in batches of 32 so a large document reports incremental progress rather than sitting silent through one giant embedding call). The re-indexing delete happens deliberately *after* a successful parse rather than at the very start of the run, specifically so that a parse failure on a re-training attempt leaves the previously-indexed version of the document intact and searchable rather than deleting it and then failing to replace it. `index_document` is registered with `max_retries=0`, and the reasoning given is that a parse failure is almost always a bad or unreadable file, not a transient problem, so a silent automatic Celery retry would just re-run expensive OCR three times over and confuse the progress display for no benefit; failures are recorded on the document for a human to see and explicitly re-trigger instead.

app/tasks/tender_pipeline.py is the largest single file in the backend and implements Stages 2 through 4 of the tender pipeline end to end: parsing, section-aware chunking, LLM extraction, evidence matching, marks reporting, and output-folder assembly, all inside one Celery task. Three things in this file are worth understanding deeply rather than skimming, because together they are the single best worked example in the codebase of a real production concurrency problem and its fix.

The first is the `run_generation` counter and `expected_generation` parameter. A Tender row can, in principle, be acted on by more than one thing at once: a worker that is still mid-stage on an old run, a user who just clicked Stop, and a user who then immediately clicked Retry, all touching the same row within moments of each other. Every write the pipeline makes to a tender row (`_advance`, called after every single stage) is expressed as an `UPDATE ... WHERE id = :id AND run_generation = :expected_generation`, checked and applied as one atomic database statement, never as a separate `SELECT` to check the value followed by a second `UPDATE` to write it (which would leave a race window between the two). `cancel_tender` and `reprocess_tender`, over in `routes/tenders.py`, both increment `run_generation` the instant a user acts, which means a worker's very next `_advance` call, whenever it happens to get around to it, finds its own generation no longer matches the row's current one, its `UPDATE` affects zero rows, and `_advance` raises `TenderCancelled` right there, cleanly stopping that specific stale worker without it ever needing to know *why* it was stopped or *what* superseded it. `run_tender_pipeline`'s own docstring explains why `expected_generation` is passed in explicitly at enqueue time rather than simply re-read from the row when the task starts running: a task can sit queued behind other work for a while, and if a second `Reprocess` bumped the counter again during that wait, a task that re-read the counter fresh at start time would wrongly believe it owned that newer generation too, and two tasks would race against each other as if they were the same run. Carrying the exact value the caller had in hand at enqueue time keeps every task permanently tied to the one specific attempt it actually represents, for its entire lifetime, no matter how long it waited in the queue first.

The second is `_fail`'s ordering, worth reading immediately after `_advance`. It calls `db.rollback()` before reading `tender.status`, specifically so that the failing stage's own half-committed work is discarded first and the in-memory tender object reflects the last stage that was actually, successfully committed, which is what makes `failed_stage` genuinely truthful rather than reporting whatever a half-applied update happened to leave behind. `_fail` is gated on the same `run_generation` check as `_advance`, for a subtle reason worth sitting with: `TenderCancelled` being raised means someone has already acted on this tender, but if that action was `Stop` immediately followed by `Retry` (rather than only `Stop`), a second, brand-new worker may already be running the new attempt by the time the first, stale worker's exception handler gets around to writing `FAILED`. Writing that unconditionally would clobber the new run's live, in-progress status the instant the old worker caught up, which is precisely the bug the whole generation guard exists to prevent, just relocated from the success path to the error path instead of being caught there too.

The third is `_run_matching`, `_compute_report`, and `_assemble_folder`'s shared, explicitly named "covered" rule, worth reading as a single unit even though it is defined three times: a match counts toward a requirement's coverage if a human has explicitly accepted it, or if it auto-matched (confidence at or above 0.85) and has not subsequently been rejected by a reviewer. `MATCH_TOP_K = 2`, `AUTO_MATCH_THRESHOLD = 0.85` and `SUGGESTED_MATCH_FLOOR = 0.60` carry an explicit comment recording that the floor and candidate count were tuned down from earlier values (a floor of 0.50 and a top-k of 5) specifically because every qualifying candidate becomes its own row a human has to click through, and the looser original settings were producing hundreds of low-value near-duplicate suggestions per tender. `_assemble_folder` writes a four-sheet Excel workbook (Requirements, Summary, Evidence Matches, Instructions) via `openpyxl`, using the tender's own verbatim wording in the `*_raw` fields wherever available rather than the normalized enum values, specifically so values like "No/Advisory" and "Financial / Pass-Fail" survive into the client-facing tracker exactly as the tender wrote them, and copies every accepted evidence file into a "Required Documents" subfolder before zipping the whole thing, rebuilding from scratch on every call (including on `finalize`) so that a reviewer rejecting a match and re-finalizing never leaves an orphaned, no-longer-relevant evidence file sitting in the package.

3.10 requirements.txt and alembic.ini

`requirements.txt` pins every dependency and its own header comment records a real decision worth understanding: the pins were taken from the fuller prototype rather than from the flat one, even though the flat prototype's own pins were individually newer across the board, because the flat prototype's package list was a strict subset that never included JWT handling, password hashing, email validation or the Anthropic client at all, none of which its stubbed-out authentication ever needed. Mixing the newer individual version numbers into the larger, real package set would mean shipping a specific combination of versions that had literally never been run together by anyone, which is a risk worth avoiding even when every individual version bump looks harmless in isolation; the comment states plainly that any future bump should happen deliberately, together with the test suite, rather than as an incidental side effect of copying a requirements file from somewhere else.

`alembic.ini` is Alembic's own configuration file and is notable mainly for what it deliberately leaves blank: `sqlalchemy.url` is empty, with a comment explaining that `alembic/env.py` overwrites it at runtime from `Settings.DATABASE_URL`, so the actual database connection string lives in exactly one place, the `.env` file, and is never duplicated into (or accidentally committed inside) a config file that would otherwise sit in version control.

4. Patterns worth reusing in your own projects

- **Ticket-based authentication for WebSockets.** A browser's WebSocket constructor cannot set an Authorization header, so putting a long-lived bearer token in the connection URL leaks it into browser history and proxy logs for as long as that token stays valid. Instead, mint a random, single-use, short-TTL ticket over an ordinary authenticated HTTP call, scope it to the exact resource the socket will connect to, and redeem it atomically (Redis GETDEL, not a GET followed by a DELETE) the moment the socket connects. This generalizes to any transport that cannot carry your normal credential.
- **Tombstone files instead of silent deletion during a merge.** When two versions of a module exist after combining codebases and only one is meant to run, do not simply delete the loser and trust that nobody references it from memory, and do not just leave it sitting there unused either. Replace its body with a clear docstring naming the live replacement and the concrete reason the old version cannot simply be revived, followed by a `raise ImportError(...)`. A stray import then fails loudly at the exact line that would otherwise have silently misconfigured the app, rather than failing mysteriously somewhere else, or worse, not failing at all.
- **A generation counter for cooperative cancellation.** When a long-running background job can be raced by a user action (stop, then immediately retry) or by itself (a stale worker still finishing while a newer attempt has already started), give the row a monotonically increasing counter. Bump it the instant the user acts, capture its value once when a worker task starts, thread that captured value through every subsequent write as part of the write's own WHERE clause, and treat a write that affects zero rows as "I have been superseded, stop." This is a general, robust pattern for cooperative cancellation of anything running outside the request/response cycle, and it does not depend on the worker ever being told explicitly that it was cancelled.
- **Two-tier model selection for cost.** Not every LLM call needs your most capable, most expensive model. Mechanical, high-volume, "extract this exact field verbatim" work (this backend's tender extraction, metadata tagging, and section classification) can run reliably on a cheap, fast model, while work that genuinely has to weigh evidence and write connected prose (a grounded question-answering feature) justifies a more expensive one. Make the model a named setting per call site rather than one single global model constant, so the tradeoff is a one-line configuration change rather than a code change.
- **Keep the request/response schema separate from the database schema.** A Pydantic (or equivalent) schema and an ORM model can look almost identical and still be doing two different jobs. Letting them diverge on purpose, rather than collapsing them into one class, is what lets your database evolve (new internal columns, a changed storage layout) without automatically changing your API, and lets your API evolve without forcing a migration.
- **Persist progress state, do not only stream it.** A WebSocket delivers events to whoever happens to be listening right now; it says nothing to a client that connects a minute late or reloads mid-run. Anywhere you stream progress, also write the latest snapshot somewhere durable (a database row, or a cache key with a sensible TTL) and always serve that snapshot first, before subscribing to new events, on every connection. Otherwise "reconnect" quietly becomes "start over from a blank screen."
- **Local, dependency-free defaults where they exist.** Choosing a locally-run embedding model instead of a second hosted API removed an entire external dependency (a second API key, a second point of failure, a second thing that can rate-limit you) at the cost of a one-time model download and some CPU time. When a capability genuinely can run locally without materially hurting quality, that is usually worth doing before reaching for another external service, especially one you would otherwise be adding purely to

save a small amount of infrastructure.

- **A dedicated, narrow health check per external dependency.** A generic “is the process alive” health check tells you almost nothing about why a specific downstream integration (here, the LLM provider) is failing. A small endpoint that exercises the exact client, URL, and call shape your real code path uses, and reports the underlying error verbatim, turns a “why is this mysteriously broken” investigation into a five-second check, and is worth building for any external dependency your app cannot function without.
- **Write down why you removed something, not just that you removed it.** Several files in this codebase (the ANTHROPIC_USER_AGENT history in `core/config.py`, the OCR path fix in `pdf_extraction.py`, the tender-metadata *_raw columns) keep a short comment explaining a decision that is no longer visible in the code itself, specifically because the reasoning would otherwise be lost the moment the old code was deleted, and a future reader (including a future version of yourself) would have no way to know the removal was deliberate rather than an oversight waiting to be “fixed” back in.
- **Grep before you trust a docstring.** This backend has at least one service module (`support_email.py`) that is fully implemented, carefully documented, and describes a real feature, but is never actually called from any live code path; the feature it describes is implemented a different way entirely, on the frontend. A confident docstring is not proof that the code it describes runs. Before you rely on what a module says about itself, especially in a codebase you did not write, check who actually imports and calls it.

VR-Nexus Frontend, Explained

This document walks through the frontend of VR-Nexus, a Tender Intelligence and Evidence Matching platform. The stack is React 19, Vite, TypeScript, React Router 7, and Tailwind CSS v4. The frontend talks to a FastAPI backend two ways: ordinary REST calls that go through a Vite dev proxy (so the browser only ever calls its own origin, no CORS needed), and two authenticated WebSocket connections that stream live progress for the tender-analysis pipeline and for evidence-library indexing jobs.

The request lifecycle, end to end, looks like this. The browser loads `index.html`, which pulls in `src/main.tsx`. `main.tsx` applies the saved theme before anything renders (so there is no flash of the wrong theme), fires off a background session-bootstrap check, and mounts `App.tsx`. `App.tsx` wraps the whole tree in `AppProviders` (currently just `BrowserRouter`) and renders `AppRouter`, which is the single route table in `src/app/router.tsx`. React Router matches the current URL against that table and picks a layout: `DashboardLayout` for anything inside the authenticated app, `AuthLayout` for the login and password screens. Inside the matched layout, a page component from `src/pages/` renders. That page component almost always uses a custom hook (`useAsyncData`, or the more specialized `useJobProgress` / `useTenderProgress`) to fetch its data. The hook calls a function in `src/services/`, which is a thin wrapper around `src/lib/apiClient.ts`, which does the actual `fetch()` call. Data that needs to be shared across unrelated parts of the tree (the signed-in user, the theme) lives in a Zustand store under `src/store/`, not in the page component. The page then renders a tree of components from `src/components/`, some of which (the two progress-tracking pages, `ProcessingPage` and `TenderProcessingPage`) open a WebSocket of their own via the progress hooks, so the browser gets pipeline updates pushed to it instead of having to poll.

One correction worth stating up front, because it is easy to assume otherwise from the phrase “React Router 7”: this codebase does **not** use the newer data-router APIs (`createBrowserRouter`, `loaders`, `actions`). It uses the classic, component-based `<Routes>` / `<Route>` API wrapped in a plain `<BrowserRouter>`. Data fetching is done entirely in hooks inside components, not in router loaders. That is a legitimate, well-supported way to use React Router 7, and it is what this codebase actually does, so that is what this document describes.

Core concepts for beginners

A React component is just a JavaScript (here, TypeScript) function that returns a description of some UI, written in JSX (HTML-looking syntax embedded in the function body). React calls that function again whenever the data it depends on changes, and efficiently patches the real DOM to match. `function Button({ children }) { return <button>{children}</button> }` is a complete, valid component. Everything in `src/pages/`, `src/components/`, and `src/Layout/` is a component like this, just larger.

Hooks are functions, always starting with `use`, that let a component “hook into” React features: `useState` for a piece of data that changes over time and should trigger a re-render when it does, `useEffect` for running code in response to a component appearing, updating, or being removed (side effects like subscriptions and network calls), `useRef` for a mutable value that does *not* trigger a re-render when it changes (a DOM node reference, or a value you need to remember across renders without re-rendering because of it). A **custom hook** is simply a function, also named `useSomething`, that calls React’s built-in hooks internally and packages up some reusable piece of logic. This codebase writes three custom hooks worth understanding before reading anything else: `useAsyncData` (load some data, track loading/error/success, allow a refetch, keep the old data visible while a background refresh happens), `useJobProgress` and `useTenderProgress` (open a WebSocket, listen for progress frames, fall back to polling if the socket won’t stay open). Writing these once and reusing them means

every page that loads data behaves the same way under slow networks, errors, and cancelled navigations, instead of every page reinventing its own ad hoc loading state.

React Router's nested layout routes let one route render a shared shell and an inner `<Outlet />` where a child route's own component appears. VR-Nexus uses this three levels deep in places: `DashboardLayout` renders the sidebar, header, and footer for the entire authenticated app and puts an `<Outlet />` in the middle; `DocumentsLayout` (nested inside that) renders the "Library / Upload / Processing" tab bar for the Documents section and puts its own `<Outlet />` inside that; the actual `DocumentsOverviewPage`, `UploadDocumentsPage`, or `ProcessingPage` renders inside that innermost outlet. The payoff is that the sidebar and the tab bar are written once, not duplicated on every page, and swapping tabs does not remount the outer shell.

A service module is this codebase's name for a file in `src/services/` whose whole job is to know the shape of one backend resource's HTTP endpoints and expose plain async functions (`listTenders()`, `getTender(id)`, `uploadDocument(file)`) that return typed data. Nothing else in the app is allowed to call `fetch()` directly or know a URL path; components and hooks call a service function and get back a typed object or throw a typed `ApiError`. This is the layer that would need to change if the backend's routes moved, and it is the layer you read first to understand "what can this app actually do to the server."

Zustand, confirmed by `import { create } from 'zustand' in authStore.ts and themeStore.ts, is a small state-management library for data that many unrelated components need to read or change: who is signed in, what the current theme is. A Zustand store is created once with create<State>((set, get) => ({ ...initialState, someAction: () => set(...) }))) and any component can call the resulting hook, useAuthStore((state) => state.user), to subscribe to just the slice of state it needs; only components that read a field that actually changed re-render. This is different from a component's own useState, which is private to that component and its children, and it is different from prop drilling, which is passing a value down through many layers of components that do not themselves need it. The rule of thumb in this codebase, and a good one generally, is: local state stays in the component (useState); state genuinely shared across the tree, or state that must persist across a page navigation without being refetched, goes in a store.`

TypeScript models are the files in `src/models/`. They are not database models and they render nothing; they are type and constant definitions that describe the exact shape of data going over the wire to and from the FastAPI backend (mirroring its Pydantic schemas), plus a handful of pure helper functions for normalizing or classifying that data. The payoff of writing these out by hand is that a typo in a field name, or sending a field the backend does not expect, is caught at compile time in your editor, months before it would otherwise surface as a confusing runtime bug or a silent undefined.

A guard, or protected-route, component is a small wrapper component that renders its children only under some condition and redirects otherwise. `RequireAuth` wraps every authenticated route and redirects to `/login` if nobody is signed in; `RedirectIfAuthenticated` wraps the login and register pages and redirects *away* from them if someone already is signed in. Guards keep that check in one place instead of every page re-implementing "if not logged in, redirect."

Why a WebSocket hook is separate from the REST service layer: a normal HTTP request-response cycle is a poor fit for "tell me every few seconds how far along this multi-minute background job has gotten." A WebSocket is a single long-lived connection the server can push messages down at any time, which is the right primitive for progress updates. But WebSockets in the browser cannot carry an Authorization header the way `fetch()` can, connections can drop for reasons that have nothing to do with the job itself (a laptop sleeping, a flaky network), and the calling component still needs an ordinary hook-shaped API (`{ progress, status }`) rather than raw socket events. `useJobProgress` and `useTenderProgress` exist to absorb all of that: minting a short-lived auth ticket over HTTP first, opening the socket, pars-

ing frames, falling back to polling the REST endpoint if the socket cannot be sustained, and cleaning up correctly when the component unmounts.

File-by-file walkthrough

Root configuration

package.json pins the dependency set: React 19.2.8 and react-dom 19.2.8, react-router-dom ^7.0.0, zustand ^5.0.2, pdfjs-dist pinned to the exact version 5.4.296 (not a caret range) alongside react-pdf ^10.1.0, react-dropzone ^14.3.8, recharts ^2.15.0, Tailwind ^4.0.0 with its @tailwindcss/vite plugin, Vite ^8.2.0, and TypeScript ^5.9.0. The build script runs `npm run typecheck` before `vite build`, so a type error fails the build rather than shipping. pdfjs-dist being pinned exactly, while everything else uses a caret range, is deliberate: react-pdf bundles a PDF.js “worker” script whose version must match the pdfjs-dist package’s API version exactly, or PDF rendering breaks with a version-mismatch error at runtime, so this one dependency cannot float to whatever a caret range would resolve to.

vite.config.ts registers the React and Tailwind v4 Vite plugins (Tailwind v4 is configured as a Vite plugin here, not as a PostCSS plugin, so there is no `tailwind.config.js` or `postcss.config.js` anywhere in the project), sets up the `@import` alias to point at `./src` (matched by a corresponding `paths` entry in `tsconfig.app.json`), and configures the dev proxy: any request to `/api/*` is forwarded to `http://127.0.0.1:8000`, and any request to `/ws/*` is forwarded as a WebSocket to `ws://127.0.0.1:8000`. A comment in the file explains why `127.0.0.1` and not `localhost` is used for the proxy target: Node can resolve `localhost` to the IPv6 loopback address `::1`, and if the backend is only bound to the IPv4 loopback, that resolves to a connection refused error that has nothing to do with the actual proxy configuration being wrong.

index.html is a minimal Vite entry HTML file: an inline data-URI SVG favicon (a red hexagon, matching the brand badge used elsewhere in the UI, chosen specifically so the browser never makes a separate network request for a favicon file that might 404), Google Fonts preconnect and stylesheet links for Sora (used for headings and buttons, referenced from CSS as `--font-display`) and Inter (used for body text and inputs, `--font-sans`), a `<div id="root">`, and a single `<script type="module" src="/src/main.tsx">` that boots the app.

src/ root

src/App.tsx is intentionally trivial: it renders `<AppProviders><AppRouter /></AppProviders>` and nothing else. Keeping the true root this thin means there is exactly one obvious place to add a new app-wide wrapper later (a query client, a toast host) without touching routing logic.

src/main.tsx is the actual entry point. Before React ever renders anything, it calls `applyTheme(getStoredTheme())` synchronously, which reads the saved theme preference from `localStorage` and stamps the corresponding `data-theme` attribute onto `<html>`. Doing this before the first paint, rather than inside a React effect after mount, is what prevents a “flash of wrong theme”: if it happened after mount, a dark-mode user would see a flash of the light theme for one frame on every load. Immediately after, it calls `void useAuthStore.getState().bootstrap()`, deliberately not awaited (the `void` operator marks that on purpose) and deliberately placed here rather than inside a `useEffect` in `App.tsx`. The reasoning, per the code’s own comments, is that React’s Strict Mode intentionally double-invokes effects in development to surface bugs, and a token-refresh call that runs twice in quick succession during bootstrap could race against itself; running it once, here, at true module-load time, sidesteps that.

src/vite-env.d.ts is a small ambient type declaration file. It narrows Vite's built-in `ImportMetaEnv` type (which otherwise allows any string key) to also specifically know about `VITE_API_BASE_URL?: string`, the one environment variable this app reads (and in development it is typically empty, since requests go through the dev proxy instead of an absolute URL).

src/index.css is where Tailwind v4's `@theme` directive defines the app's design tokens directly in CSS (a v4 feature; there is no separate JS theme config file). It sets `--font-display` (Sora) and `--font-sans` (Inter) with system-font fallback stacks, brand colors (brand-500 is the signature red, #e8151b), a dark "ink" palette used for the sidebar, and semantic tokens like `surface`, `field`, and `hairline` that are defined once for light mode and then redefined under a `[data-theme='dark']` selector so the same class names (`bg-surface`, `border-hairline`) automatically look right in both themes. A `@custom-variant dark` directive ties Tailwind's `dark: variant` to that same `data-theme="dark"` attribute rather than Tailwind's default (which watches for a `.dark` class on `<html>`) or the OS-level `prefers-color-scheme` media query directly; this is what lets `theme.ts` implement an explicit light/dark/system toggle rather than being locked to the OS setting.

src/app/

src/app/router.tsx is the single source of truth for every URL in the app, using the classic `<Routes> / <Route>` component tree (not `createBrowserRouter`). Public routes (`/login`, `/register`) are wrapped in `RedirectIfAuthenticated`; the `forgot-password` and `reset-password` routes are not gated at all, since a signed-in user might still need to reset a password. Every authenticated route is nested inside one `<Route element={}<RequireAuth><DashboardLayout /></RequireAuth></>` block. Inside that block, the `Documents` section and the `Tender Analysis` section are each further nested inside their own layout route (`DocumentsLayout`, `TenderAnalysisLayout`) so their `index/upload/processing` tabs share a tab bar. Detail and review pages for a specific tender or document (matched by an ID in the URL, using patterns like `TENDER_DETAIL_PATTERN`) are deliberately *not* nested as another tab inside the `Tender Analysis` layout; they sit as sibling routes directly under `DashboardLayout`. The reasoning given in the surrounding code is about not implying an affordance that does not exist: nesting them as a fourth tab would visually suggest you can tab away from a tender's review screen and tab back, when the actual navigation model is "open a specific tender, then go back to the list." A catch-all `<Route path="*">` redirects anything unmatched to `/login`.

src/app/providers.tsx exports `AppProviders`, which currently only wraps its children in `<BrowserRouter>`. A code comment marks it explicitly as the single place future app-wide providers (a data-fetching cache/query client, a toast notification host) should be added, so that `App.tsx` itself never needs to change shape again.

src/app/guards.tsx defines the two route guards. `RequireAuth` checks `isAuthenticated` from the auth store; if false, it renders `<Navigate to="/login" state={{ from: pathname + search }} />` so the login page can send the user back to where they were headed after they sign in. It optimistically trusts a session that was restored from storage before the backend has had a chance to verify the token is still valid, on the reasoning that showing the UI immediately (and letting a background 401 correct course if the token turns out to be stale) is a better experience than blocking on a network round trip on every navigation. `RedirectIfAuthenticated` is the mirror image for the login and register pages, but with an important asymmetry: it waits for `isBootstrapping` to become false before deciding whether to redirect, so a user with a genuinely valid stored session is not shown a flash of the login form while that session is still being verified. The file also exports `readReturnPath(state)`, a small guard function that only trusts a `from` path if it starts with a single `/` and not `//`, which blocks a crafted redirect URL from bouncing the user to an external site (an open-redirect vulnerability) after login.

src/constants/routes.ts defines the `ROUTES` object (an `as const` object of every static path

in the app) plus, for the three routes that carry a dynamic ID in their URL, a matched pair of exports: a `_PATTERN` string for React Router to match against (e.g. `TENDER_DETAIL_PATTERN = '/tenders/:tenderId'`) and a builder function (`tenderDetailPath(id)`) that other code calls to construct an actual link, so no component ever hand-writes a template string for one of these URLs and risks a typo diverging from the route table.

src/lib/

src/lib/apiClient.ts is the lowest layer of the network stack and the only file in the app that calls `fetch()` directly. It exports an `ApiError` class carrying a message, an HTTP status, an optional structured detail payload from the backend, and an `isOffline` flag; `readMessage()` is a small helper that unwraps FastAPI's several different error-body shapes (a plain string, a `{ detail: string }`, or a `{ detail: [...] }` validation-error array) into one consistent message. To avoid a circular import with `authStore.ts` (the client needs to read the current access token and needs to trigger a refresh on a 401, but the auth store needs to call the client to make its own requests), the file exposes two injection seams, `setAuthProvider(fn)` and `setUnauthorizedHandler(fn)`, which `authStore.ts` calls once at its own module load time to hand the client a getter and a callback. `refreshOnce()` implements a single-flight pattern: if several requests hit a 401 at nearly the same moment, only the first triggers an actual token refresh, and the rest await that same in-flight promise instead of each firing their own redundant refresh call. The core `request<T>()` function does the actual fetch, retries exactly once on a 401 (guarded by an `isRetry` parameter so a second failure does not loop forever), and the exported `api` object exposes the verbs the rest of the app calls: `get`, `getBlob` (for binary responses like PDFs), `postJson`, `postForm`, `postEmpty`, `patchJson`, `delete`. `websocketUrl(path)` builds a `ws://` or `wss://` URL from a relative path (matching the protocol of the current page), and `errorMessage(error)` is a small safe extractor used throughout the UI to turn any caught error into a displayable string.

src/lib/formatting.ts collects small, pure display-formatting functions so the same date or number never gets formatted two different ways in two different components. `formatShortDate` / `formatLongDate` deliberately use a fixed day-month-year (en-GB) format rather than letting the browser's locale decide, because the product's audience is Pakistani and a locale-auto format would silently flip to month-day-year for a US-locale browser. `formatRelativeTime` ("just now", "5 min ago", "2 days ago") is hand-written rather than using the built-in `Intl.RelativeTimeFormat`, again for consistent wording across browsers. `formatBytes` and `formatGigabytes` handle file-size display (1024-based; kilobytes and plain bytes show no decimal place, megabytes and gigabytes show one). `formatCount`, `formatSignedPercent`, `formatInitials`, `formatRole`, and `formatFirstName` round out small display needs used across the dashboard and account screens.

src/lib/theme.ts defines the `Theme = 'light' | 'dark' | 'system'` type, `getStoredTheme` / `storeTheme` (both wrapped in `try/catch` since `localStorage` can throw in some private-browsing contexts), `prefersDark()` (reads the OS-level media query), `resolveTheme()` (turns 'system' into a concrete 'light' or 'dark' by checking `prefersDark()`), and `applyTheme()`, which stamps the resolved theme onto `<html data-theme="...">` and also sets the native `colorScheme` CSS property so browser-drawn UI (scrollbars, form controls) matches too.

src/lib/validation.ts holds the client-side form-validation functions: `validateName`, `validateEmail` (a deliberately loose regex, on the stated principle that the server is the actual authority on what counts as a valid email, and the client check exists only to catch obvious typos before a round trip), `validateNewPassword` (which measures length in bytes, using `TextEncoder`, not characters, because the backend hashes passwords with `bcrypt` and `bcrypt` silently truncates at 72 bytes; a password with multi-byte Unicode characters could pass a character-count check but still exceed the real byte limit, so the error message distinguishes the two), `validatePasswordPresent`, `validateConfirmPassword`, `validateOptionalPhone`, and

measurePasswordStrength (a 0 to 4 score used only to show feedback to the user, not to block form submission).

src/lib/pdfWorker.ts does the one-time setup react-pdf needs: it points pdfjs.GlobalWorkerOptions.worker at the PDF.js worker script using `new URL('pdfjs-dist/build/pdf.worker.min.mjs', import.meta.url)` (a pattern Vite understands and bundles correctly), imports the `Annotation-Layer.css` and `TextLayer.css` stylesheets PDF.js needs for text selection and link overlays to render correctly, and exports a constant `PDF_OPTIONS = { isEvalSupported: false }`. That options object has to be declared once at module scope rather than written inline where it is used, because react-pdf compares the options prop it receives by object identity between renders; a fresh object literal on every render would look like a “new” options value each time and cause the PDF to be needlessly reloaded.

src/models/

These files hold no rendering logic. They are the TypeScript description of exactly what the backend sends and expects, plus small pure functions for normalizing or deriving values from that data.

src/models/index.ts is a barrel file: `export * from './auth'`, so other files can import auth-related types from `@/models` instead of reaching into `@/models/auth` directly. (Notably, the other model files, `dashboard.ts`, `documents.ts`, `tenders.ts`, are imported directly by their own paths elsewhere in the app rather than being re-exported from this barrel.)

src/models/auth.ts defines `USER_ROLES` and the derived `UserRole` union type (as a readonly array plus `typeof USER_ROLES[number]`, rather than a TypeScript enum; see the closing patterns section for why), field-limit constants that mirror backend validation exactly (`NAME_MAX_LENGTH = 255`, `PASSWORD_MIN_LENGTH = 8`, `PASSWORD_MAX_LENGTH = 72` matching bcrypt’s byte limit, `PHONE_MAX_LENGTH = 32`, `COMPANY_MAX_LENGTH = 255`), `normalizeUserRole` and `normalizeUser` (which defend against a role coming back from the server in unexpected casing or an unrecognized value by defaulting to the least-privileged role, ‘user’, rather than trusting unknown input), the wire request/response types `LoginRequest`, `RegisterRequest`, `RefreshRequest`, `ForgotPasswordRequest`, `ResetPasswordRequest`, `MessageResponse`, `User`, `TokenResponse`, the form-only types `LoginFormValues` and `RegisterFormValues`, the converters `toRegisterRequest` / `toLoginRequest` (which trim whitespace and convert blank optional fields to null before sending), `AuthSession`, and `AuthErrorKind` (a union of `invalid` | `expired` | `locked` | `disabled` | `conflict` | `validation` | `offline` | `unknown`) with its companion `AuthFailure` type, which is what lets `AuthAlert.tsx` show a different icon and message for “wrong password” versus “account locked” versus “you’re offline” instead of one generic error string.

src/models/dashboard.ts is different in kind from the other model files: everything in it is a UI-only type, not a wire type, because there is no single dashboard endpoint on the backend. `SummaryStat`, `Project` / `ProjectStatus`, `ActivityEntry` / `ActivityKind`, `AnalysisPoint` / `AnalysisRange`, `StorageUsage`, `AssistantSuggestion`, `QuickAction`, and the aggregate `DashboardData` describe the shapes `DashboardPage.tsx` assembles on the client by combining data from the tender and document services.

src/models/documents.ts is the largest model file and mirrors the evidence-library backend closely. `DOCUMENT_CATEGORIES` and `CATEGORY_LABELS` define the fixed set of document categories and their display names; `DOCUMENT_FILE_TYPES` and `DROPZONE_ACCEPT` map accepted MIME types to file extensions for the upload dropzone, matched to what the server will actually accept, and `MAX_UPLOAD_MB = 100` caps upload size client-side before a wasted round trip. `fileTypeFromName()` infers a file’s type from its extension. `TRAINING_STATUSES` enumerates a document’s indexing lifecycle (queued, parsing, tagging, embedding, indexed, failed); `IN_FLIGHT_STATUSES` and `isInFlight()` identify the subset that means “still working.”

JOB_STAGES, STAGE_LABELS, and STAGE_SEQUENCE describe the five real processing stages a job walks through in order (failed is deliberately excluded from STAGE_SEQUENCE, since it is an outcome that can interrupt any stage, not a stage of its own); stageStates() and isTerminalStage() derive per-stage UI state (done / active / pending / failed) from a job's current status, which is what ProcessingTimeline.tsx renders directly. The wire types LibraryDocument, DocumentChunk, LibraryDocumentDetail, IndexJob, and ProgressEvent describe what the backend actually returns. DuplicateMatch / DuplicateCheckResult and UploadResponse / TrainResponse cover the upload flow. CategoryCounts, LibraryStats, StatsRollup, and rollUpStats() support the summary tiles on the library overview page. UploadMetadata, EMPTY_UPLOAD_METADATA, and toUploadFormData() describe the metadata form fields attached to an upload; the field names in toUploadFormData() are pinned using a keyof constraint against the request type, so a renamed backend field would fail to compile here rather than silently sending the old field name forever. A comment block in the file explicitly documents which fields are deliberately *not* modeled (size_bytes, uploaded_by_name, summary, file_url) and why, which is a genuinely useful habit: it tells the next reader that an omission was a decision, not an oversight. PendingUpload models an in-progress upload queue item; documentTitle(), METADATA_FIELDS, and metadataTags() are small display helpers; LibrarySearchHit, LibrarySearchResponse, and LibraryAnswer describe the search and ask (RAG) endpoint responses used by AiAssistantPage.

src/models/tenders.ts mirrors the tender-analysis pipeline. TENDER_STATUSES lists all eleven possible statuses (nine pipeline stages plus the two terminal outcomes finalized and failed); PIPELINE_STAGES is the nine in-flight ones. TENDER_STATUS_LABELS, STAGE_LABELS, and STAGE_DESCRIPTIONS supply display text. IN_FLIGHT_STATUSES, isInFlight, isSocketTerminal, and isSettled classify a status for UI purposes; notably, isSocketTerminal treats ready_for_review as *not* terminal for the WebSocket, since the socket in useTenderProgress deliberately stays open through the human review stage, only closing at finalized or failed. stageStates() mirrors the equivalent function in documents.ts for TenderTimeline.tsx. REQUIREMENT_STATUSES and EVALUATION_IMPACTS describe how an extracted requirement is classified after evidence matching; requirementPriority() derives a display priority from that. MATCH_TYPES encodes the three-way bucketing of an evidence match by its similarity score: auto for a similarity of 0.85 or above, suggested for 0.50 to 0.84, missing below 0.50. MATCH_REVIEW_STATUSES, MatchReviewAction, and MatchReviewUpdate support the human-in-the-loop review workflow. The wire types TenderUploadResponse, TenderListItem, TenderDetail, TenderUpdate, Requirement, EvidenceMatch, ImpactBreakdown, TenderReport, and TenderChunk describe the backend's responses; TenderProgressFrame / TenderProgress and their normalizers progressFromFrame() / progressFromDetail() let the progress hook and a plain REST-loaded tender detail both be converted into the same shape for the timeline component to render, regardless of which source the data came from. TENDER_DROPZONE_ACCEPT narrows the tender upload dropzone to PDF only, unlike the library dropzone's broader accept list. TenderUploadMetadata, EMPTY_TENDER_METADATA, tenderTitle(), and hasReviewableAnalysis() finish out the file.

src/store/

src/store/authStorage.ts is a small, hand-rolled persistence helper, deliberately not using Zustand's built-in persist middleware. The reason is the "Remember me" checkbox on login: whether a session should survive a browser restart (localStorage) or only last the current tab session (sessionStorage) is a choice made dynamically at sign-in time, and persist is built around one fixed storage backend configured up front, not a per-login choice between two. StoredSession and RestoredSession are the relevant types; isStoredSession() is a runtime type guard that validates whatever comes back out of storage before trusting it (since storage content is not something TypeScript's compile-time types can actually guarantee, a user or a bug could have put anything there). readStoredSession() checks sessionStorage first and

falls back to `localStorage`; `writeStoredSession()` writes to the correct one for the current “remember me” choice and explicitly clears the other, so a session cannot end up duplicated in both; `clearStoredSession()` wipes both on logout.

src/store/authStore.ts is the confirmed Zustand store for authentication (import { create } from 'zustand'). Its state includes `user`, `accessToken`, `refreshToken`, `remember`, `isSubmitting`, `isBootstrapping`, `failure`, and `notice`; its actions are `login`, `register`, `logout`, `refreshSession`, `bootstrap`, `clearFailure`, `clearNotice`, and `setNotice`. At module load time (outside the `create()` call, so it runs exactly once when the module first imports), it calls `readStoredSession()` to seed the store’s initial state, which is what lets `RequireAuth` optimistically trust a restored session before any network round trip confirms it is still valid. `SESSION_EXPIRED` is a constant `AuthFailure` value reused wherever an expired session needs to be reported consistently. At the bottom of the file, `setAuthTokenProvider(() => useAuthStore.getState().accessToken)` and `setUnauthorizedHandler(() => useAuthStore.getState().refreshSession())` are called to plug this store into `apiClient.ts`’s injection seams, which is the resolution of the circular-dependency problem described under `apiClient.ts` above. The file also exports two selector functions, `selectIsAuthenticated` and `selectIsAdmin`, meant to be passed into `useAuthStore(selector)` calls elsewhere so that a component subscribes to exactly the derived boolean it needs rather than the whole user object.

src/store/themeStore.ts is the second, much smaller Zustand store. `ThemeState` holds `theme` (the user’s stated preference: `light`, `dark`, or `system`), `resolved` (the concrete light-or-dark value actually applied), and a `setTheme` action that does three things in one call: persists the new choice via `storeTheme()`, applies it to the DOM via `applyTheme()`, and updates the store’s own state so subscribed components (like the theme toggle button) reflect the new active state. A module-level `matchMedia('(prefers-color-scheme: dark)').addEventListener('change', ...)` listener re-invokes `setTheme('system')` whenever the OS-level preference changes, but only takes effect if the user’s stored preference is currently `'system'`; a user who has explicitly chosen light or dark is not overridden by an OS change.

src/hooks/

src/hooks/useAsyncData.ts is the general-purpose data-fetching hook used by nearly every page in the app. Called as `useAsyncData(fetcher, deps)`, it returns { `status`, `data`, `error`, `offline`, `notFound`, `isRefreshing`, `refetch` }. Internally it creates a fresh `AbortController` for each run and aborts the previous one if the dependencies change or the component unmounts before a request finishes, which prevents a slow, stale response from a previous navigation overwriting fresher state. It keeps the `fetcher` function in a `ref` rather than a plain closure so that passing a new (but logically identical) function on every render does not re-trigger the effect; only the `deps` array controls when a `refetch` happens, plus an internal `reloadToken` counter that `refetch()` bumps to force a rerun on demand. It distinguishes a true first load (blank state, shows a loading skeleton) from a `refetch` of already-loaded data (keeps the previous data visible and sets `isRefreshing` instead of clearing the screen), and if a `refetch` fails, it keeps the stale data on screen rather than replacing a working view with an error, while still surfacing the error and an `offline` flag for the caller to show as a banner.

src/hooks/useJobProgress.ts tracks one library-document indexing job over its lifetime, cycling through the states `idle`, `live`, `polling`, `offline`, and `closed`. It first calls `mintJobWSTicket()` (an HTTP POST that returns a short-lived, single-use ticket, since the WebSocket handshake itself cannot carry an `Authorization` header), then opens the socket at `jobProgressUrl(jobId, ticket)`. It filters out ping frames and a distinct error-frame shape the server sends. If the socket fails three times in a row, it gives up on the socket and falls back to polling `getJob()` every two seconds instead, so a job’s progress is never permanently stuck

just because a WebSocket connection would not stay open (a corporate proxy blocking WebSocket upgrades, for instance). It tracks `lastRealStage` separately so that if a failure frame arrives without its own stage information, the UI can still correctly attribute the failure to whichever stage was last actually in progress. On cleanup, it explicitly nulls out the socket's event handlers before calling `.close()`, because closing a socket fires its own `onclose` handler, and without nulling the handlers first, that teardown-triggered `onclose` would itself try to start a new polling fallback right as the component is unmounting.

src/hooks/useTenderProgress.ts is the sibling hook for the tender-analysis pipeline, sharing the same overall shape (`idle / live / polling / offline / closed`) and the same ticket-then-socket pattern, but with different termination rules suited to a pipeline that includes a human review step: `ready_for_review` is explicitly *not* treated as socket-terminal, so the connection stays open through the review stage (in case, for example, a re-analysis is triggered while the review screen is still open); the true terminal states are only `finalized` and `failed`. This socket also does not receive ping frames the way the job-progress socket does, a small but real protocol difference between the two backend endpoints that the hook accounts for.

src/services/

src/services/authService.ts defines `AUTH_ENDPOINTS`, an object of the auth API's URL paths, and exposes `authService.login`, `register`, `refresh`, `me`, `forgotPassword`, and `resetPassword`, each a thin async wrapper around a call to `api` from `apiClient.ts`. Every response that carries a user object is passed through `normalizeUser()` from the models layer before being returned, so a defensively-normalized user is the only shape the rest of the app ever sees. `classifyAuthError()` maps an `ApiError`'s HTTP status code onto one of the `AuthErrorKind` values, which is what lets the login form show "your account is locked" instead of a generic "request failed."

src/services/documentService.ts covers the evidence library. Reads: `listDocuments`, `getDocument`, `getStats`, `listStages`. Uploading: `uploadDocument`, `checkDuplicate`. Indexing: `train`, `retrainDocument`, `getJob`, `listDocumentJobs`, `deleteDocument`. `listProcessingQueue()` is a composed read with no single matching backend endpoint: it fans out five separately-filtered calls to `listDocuments` plus a `listDocumentJobs` call per document, capped at a `QUEUE_LIMIT` of 60 documents, to assemble the "what's currently processing" view. `mintJobWsTicket` and `jobProgressUrl` support the ticket-based WebSocket handshake described above. `documentImageUrl` and `documentFileUrl` build the raw backend path to a document's rendered page image or original file, but those paths are not directly usable in a plain `` or `<a href="...">`, since those requests would go out without an auth header; `fetchDocumentFileObjectUrl`, `fetchDocumentImageObjectUrl`, and `releaseObjectUrl` are the actual mechanism used, fetching the binary through the authenticated `api.getBlob()` call and turning the result into a `URL.createObjectURL()` object URL that a normal `<img>` or `<a>` tag can then point to, with `releaseObjectUrl` freeing the memory once it is no longer needed. `searchLibrary` and `askLibrary` call the retrieval and RAG-answer endpoints used by `AiAssistantPage`.

src/services/tenderService.ts mirrors `documentService.ts`'s structure for tenders. Reading: `listTenders`, `getTender`, `listRequirements`, `listMatches`, `getReport`. Uploading: `uploadTender` (a single-file, file-only request body, unlike the library upload which also carries metadata fields). Editing and review: `updateTender`, `reviewMatch`, `finalizeTender`, `cancelTender`, `reportTenderIssue`, `deleteTender`, `reprocessTender`. `mintTenderWsTicket` and `tenderProgressUrl` are the tender-side equivalents of the job-progress ticket functions. `tenderFileUrl` and `tenderDownloadUrl` are raw path builders; `fetchTenderFileObjectUrl`, `fetchTenderOutputObjectUrl`, and `releaseObjectUrl` are the authenticated-blob-to-object-URL functions used to actually display or download a tender's source PDF or its generated output document.

src/Layout/

src/Layout/AuthLayout.tsx is the shared shell for every unauthenticated screen (login, register, forgot password, reset password): a split-panel layout with a branded image on one side and the form on the other, with a hexagon badge straddling the seam between the two panels using CSS pseudo-elements. A width prop ('narrow' | 'wide') lets a given page's form panel be sized appropriately (the two-field login form needs less width than the multi-field register form).

src/Layout/DashboardLayout.tsx is the shell every authenticated page renders inside. It owns several pieces of state itself: `isSidebarOpen` (the mobile drawer's open/closed state), `isSidebarCollapsed` (the desktop rail's collapsed state, persisted to `localStorage` so it survives a reload), `menuButtonRef` (kept so focus can be programmatically returned to the menu button when the mobile drawer closes, which matters for keyboard and screen-reader users), and `scrollRef`. A `useEffect` keyed on `location.pathname` resets that scroll container back to the top on every route change, since without it, navigating from a long scrolled-down page to a new page would land the user mid-scroll on the new page's content instead of at its top. It renders `<Sidebar>`, `<Header>`, a `<main>` containing either an explicitly passed `children` prop or, more commonly, `<Outlet />` for the matched child route, and a `<Footer>`.

src/Layout/DocumentsLayout.tsx renders an `<h1>Documents</h1>` heading, a segmented control built from three `NavLinks` (Library, Upload, Processing) whose active styling comes from `NavLink`'s automatic active-class behavior, and an `<Outlet />` for whichever of the three tab pages is current.

src/Layout/TenderAnalysisLayout.tsx is the same pattern applied to the Tender Analysis section, with an "Overview / Upload / Processing" tab set instead.

src/components/dashboard/

AiAssistantPanel.tsx is the small, dark, visually inverted card on the home dashboard offering a few chip-shaped example prompts; clicking one navigates to `/ai-assistant?q=...`, handing off to the full `AiAssistantPage`.

AnalysisOverview.tsx renders a `Recharts AreaChart` of activity over time with a switchable time range, and checks `prefers-reduced-motion` before enabling the chart's animation, so a user who has asked their OS to minimize motion does not get an animated chart regardless.

Footer.tsx is the small persistent footer inside `DashboardLayout`.

Header.tsx renders the top bar, including a search box that searches across both `listTenders` and `listDocuments` results composed together client-side (there being no unified search endpoint), a `NotificationsMenu` built the same composed way, and the `ThemeToggle` control.

Panel.tsx is the single reusable card shell used across nearly every page in the app: a bordered, padded container with an optional title, description, and action slot in its header. It supports an optional collapsible mode that turns it into an accordion (used, for instance, by the three sections of `SettingsPage`), which is what makes it flexible enough to serve as both a static content card and an expandable settings section without two separate components.

PanelLink.tsx is a small link-styled row variant meant to sit inside a `Panel`.

QuickActions.tsx and **RecentActivity.tsx** and **RecentProjects.tsx** render the corresponding sections of the home dashboard, each driven by data `DashboardPage.tsx` assembles.

Sidebar.tsx is the collapsible left navigation rail. It is styled dark in both light and dark theme (the brand's navigation chrome does not flip with the rest of the UI), which is why its own internal styles use fixed white-with-opacity color values rather than the semantic text-

neutral-* tokens the rest of the app uses; those tokens are defined to invert with the theme, and the sidebar deliberately does not want to invert.

StatCard.tsx and **StatusPill.tsx** are small, generic display primitives (a labeled number tile, and a colored status badge) reused by several pages beyond just the dashboard.

StorageMeter.tsx renders a Recharts donut/pie chart of storage usage.

src/components/documents/

DocumentDropzone.tsx wraps react-dropzone to build the file drop target on the upload page, validating file type and size against the same DROPZONE_ACCEPT and MAX_UPLOAD_MB constants defined in models/documents.ts, so the client-side check mirrors what the server will actually accept rather than drifting from it.

FileGlyph.tsx picks an icon based on a file's type or extension.

JobStagePill.tsx is a small colored badge for a single indexing job's current stage.

ProcessingTimeline.tsx renders the five-node stage track for a document's indexing progress, in both a horizontal (desktop) and a stacked (mobile) orientation, driven by the stageStates() helper from models/documents.ts.

TrainingStatusPill.tsx is the equivalent compact status badge for a document's overall training/indexing status.

src/components/feedback/

AlertMessage.tsx is a generic message banner with five tone variants (info, success, warning, error, and a neutral default), correctly choosing role="alert" for tones that should interrupt a screen reader immediately versus role="status" for ones that should not.

AuthAlert.tsx is a thin wrapper specifically for auth errors: it maps an AuthFailure's kind (from models/auth.ts) onto the right tone, icon, and title for AlertMessage to render, so LoginPage and RegisterPage do not each need their own copy of that mapping logic.

DataState.tsx holds the reusable three-state async UI pattern used throughout the app: LoadingRows (a skeleton placeholder), ErrorBlock (an error display with an optional retry button and special-cased copy for an offline error), StaleDataNotice (a small inline banner shown when a background refresh has failed but the previously-loaded data is still being shown), and AsyncSection, which composes the three of those around a piece of content based on a use-AsyncData result, so a page can wire up its loading/error/success states in a few lines instead of hand-writing the same if (status === 'loading') ... branching on every page.

src/components/tender/

ComplianceMatrix.tsx renders an editable table of requirements, clauses, owners, evidence, and status; user edits to fields the backend does not itself track are persisted to localStorage, keyed per tender ID, so a reviewer's manual annotations survive a reload without needing a backend field for them.

EvaluationSimulator.tsx is an interactive what-if scoring tool with a hand-drawn conic-gradient ring visualizing the simulated score, letting a reviewer adjust assumptions and see how the technical evaluation score would move.

ReassignDialog.tsx uses the native <dialog> element (see the closing patterns section) and only fetches the evidence library's document list while the dialog is actually open, avoiding an unnecessary request every time the review page itself loads.

RequirementReviewCard.tsx is the core unit of the tender review workspace: one card per extracted requirement, showing its matched evidence and letting a reviewer accept, reassign, or flag it. It exports two standalone helper functions, `requirementCoverage()` and `isNarrativeBoilerplate()`, that `TenderReviewPage.tsx` imports directly to split requirements into an “actionable” list and a “narrative” list before rendering, which is a good example of a component file usefully exporting pure logic alongside its default rendering export rather than forcing that logic to be duplicated or awkwardly re-derived by the parent.

RiskScanner.tsx runs a client-side keyword and pattern match across the requirement text for seven risk categories (things like liability, SLA, and payment-term red flags), entirely in the browser with no backend call, since it is a simple text-pattern scan.

SubmissionChecker.tsx combines a manually-ticked checklist with a few automated checks, persisting checklist state to `localStorage` the same way `ComplianceMatrix.tsx` does.

TenderDropzone.tsx is the tender-specific upload dropzone, using the narrower `TENDER_DROPZONE_ACCEPT` (PDF only) from `models/tenders.ts`.

TenderStatusPill.tsx is the compact status badge for a tender’s overall pipeline status.

TenderTimeline.tsx renders the nine-node pipeline stage track, the tender-analysis equivalent of `ProcessingTimeline.tsx`.

ClarificationGenerator.tsx (present in the codebase though not called out in the original file inventory) auto-generates clarification questions from a tender’s requirements and also lets a reviewer add their own manually, exporting the resulting list as a downloaded `.txt` file.

src/components/ui/

ActionButton.tsx (and its sibling export `IconAction`) is the app’s primary button component, handling variants (primary/secondary/danger and similar), sizes, leading and trailing icon slots, and an optional `to` prop that turns the button into a React Router Link when navigation rather than a click handler is what’s needed, so callers do not have to choose between a styled button and a styled link component; one component covers both.

BrandMark.tsx renders the hexagon logo mark used in the header, the auth layout seam, and the favicon.

Button.tsx is a lower-level, more generic button primitive that `ActionButton` and other components build on.

Checkbox.tsx is the styled checkbox input used by the “Remember me” control and the various checklist components.

ConfirmDialog.tsx is a native `<dialog>`-based confirmation modal (see closing patterns section), used before destructive actions like deleting a document or a tender.

DataTable.tsx is a generic, reusable table component, typed as `DataTable<T>` with a `Column<T>` type describing each column’s header, cell renderer, and optional sort key. It supports sortable columns and row selection but deliberately has no built-in pagination, because every table it is used for in this app is backed by server-side pagination already handled by the calling page, not by a large in-memory array the table itself would need to page through. `TableEmptyState` is a matching empty-state component exported from the same file.

LabelledDivider.tsx is the small “or” divider used to separate, for example, a form from an alternate sign-in method.

TextField.tsx is the shared styled text input, wired up to show validation errors consistently.

icons.tsx exports roughly sixty hand-drawn SVG icon components sharing one common stroke configuration (width, line cap, line join), which is why the app has no icon-library

dependency in `package.json` at all: every icon used anywhere in the UI is defined directly in this one file as plain SVG paths.

surfaces.ts exports a couple of shared `className` string constants (`BRAND_SURFACE`, `QUIET_SURFACE`) bundling a gradient background and border treatment used by more than one component, so that particular visual treatment is defined in exactly one place.

src/pages/

HomePage.tsx is explicitly marked in its own comment as a temporary placeholder with an instruction to delete it once the real workspace landing experience is built; it is worth noting as-is rather than described as more finished than it is.

AboutPage.tsx (present in the codebase, not in the original file inventory) is a real, finished content page: team credits, a list of platform capabilities, and a row of technology chips naming the stack.

LoginPage.tsx, **RegisterPage.tsx**, **ForgotPasswordPage.tsx**, and **ResetPasswordPage.tsx** together implement the full authentication flow, each built from `TextField`, `Button`, `Checkbox`, and `AuthAlert`, calling into `authStore`'s actions and rendering whatever `AuthFailure` comes back.

DashboardPage.tsx builds its stats, project list, and activity feed from real calls to `listTenders` and `getStats`, with no mock or placeholder data; its `pctChange()` and `countInWindow()` helper functions compute the month-over-month trend figures shown on the stat cards.

DocumentsOverviewPage.tsx is the evidence library's listing page: server-side category and status filters combined with a client-side text search, bulk selection with batch train/delete actions, and `SummaryFigure` tiles summarizing the library's contents.

UploadDocumentsPage.tsx lays out a three-panel upload flow (a category radio-card selector, the files dropzone plus an upload queue, and a details disclosure panel for metadata), and automatically continues on to indexing once a batch of uploads completes cleanly with no errors.

ProcessingPage.tsx is the library-indexing progress screen: a focused `JobWatcher` panel for whichever job is currently being watched, a `WaitingPanel` state, and a queue list of other in-flight jobs, and is the clearest place in the codebase to see `useJobProgress` actually wired up to UI.

PdfViewerPage.tsx is one of the largest and most involved pages. `canRenderPages()` gates whether the in-browser PDF renderer should even attempt to run (some documents, or some states, are not renderable). `PdfPane` wraps `react-pdf`, measuring the container to fit the page width, providing zoom controls, and fetching the PDF itself through the authenticated object-URL pattern with a retry path if the fetch fails. `ExtractedContent` renders the document's extracted text broken into chunks with section and page markers. `ChunkFigure` lazy-loads each chunk's associated image only when it scrolls into view, using an `IntersectionObserver`, rather than loading every image up front. `DownloadFileButton` implements the fetch-then-blob-then-synthetic-anchor-click pattern needed to trigger a real file download for an authenticated resource, since a plain `<a href>` cannot carry an auth header.

TenderOverviewPage.tsx is the tender listing page, structurally similar to `DocumentsOverviewPage.tsx`, but its filters run entirely client-side, since (unlike the document library) there is no tender-stats endpoint on the backend to filter server-side against. It offers `Retry` and `Re-analyse` actions alongside `Delete` on each row.

UploadTenderPage.tsx is a single-file upload flow (a tender upload is one PDF, not a batch). Its `buildPatch()` and `metadataError()` helpers support a two-step upload-then-update-metadata flow: the file is uploaded first, and a follow-up `PATCH` call attaches the metadata fields, with

the page degrading gracefully (the upload itself is still considered a success) if that second metadata call fails.

TenderProcessingPage.tsx is the tender-side counterpart to `ProcessingPage.tsx`, built around a `TenderWatcher` focused panel driven by `useTenderProgress`, plus a `TenderErrorLog` component whose `supportGmailCompose()` helper builds a pre-filled Gmail compose URL (a `mailto/Gmail-compose` link) so a user hitting a pipeline failure can quickly email support with the error details already filled in, alongside a `reportTenderIssue()` call that also logs the issue through the API.

TenderReviewPage.tsx is the core review workspace and one of the most complex pages in the app. It loads the tender, its requirements, its evidence matches, and its report all in parallel with `Promise.all`, then splits the requirements into `narrativeRequirements` and `actionableRequirements` using the `isNarrativeBoilerplate()` helper imported from `RequirementReviewCard.tsx`. A `StatTile` summary strip shows top-line coverage numbers, and the bulk of the page is a filterable list of `RequirementReviewCard` components, with `ReassignDialog` and `ConfirmDialog` wired in for reassigning evidence and confirming destructive actions, and `Finalize` and `Download` actions completing the workflow.

TenderToolsPage.tsx hosts four analysis utilities (`ComplianceMatrix`, `EvaluationSimulator`, `RiskScanner`, `SubmissionChecker`) behind one tab bar, loading the tender's data once with `useAsyncData` and sharing it across all four tabs so switching between tools is instant rather than re-fetching per tab. Before the tabs render, it gates on the tender's status: if the tender is still uploaded, in flight, or failed, it shows an explanatory message instead of the tools, since none of the four tools has anything meaningful to analyze until a completed analysis exists.

AiAssistantPage.tsx is the frontend for two backend endpoints that otherwise had no UI: `POST /api/library/ask` (a retrieval-augmented-generation question-answering endpoint) and `GET /api/library/search` (retrieval only, no generation). Both read the same index the tender pipeline's evidence matcher reads, so an answer here is grounded in the same documents a tender match would draw from, not from the underlying model's general knowledge. The page has two modes sharing one input box: `Ask` returns a written answer plus the passages it was built from, and if the library cannot support the question, or generation is switched off entirely (no API key configured on the backend), the response comes back with `grounded: false` and the page shows the retrieved passages anyway rather than hiding them, on the reasoning that showing the closest evidence honestly is better than showing nothing. `Find` skips generation and returns ranked passages directly, which is presented as the honest fallback capability when no language model is configured at all. `dedupeByDocument()` collapses multiple chunk-level hits from the same source document down to its single best-scoring hit, since the same case study matching on three separate chunks would otherwise show as three redundant rows. The conversation itself lives only in component state (a `useState` array of turns); nothing is written to `localStorage`, and a page reload starts a fresh conversation, which the file's own comment justifies as the right default for a question box over a shared library rather than a private, worth-persisting thread.

ActivityPage.tsx composes a single reverse-chronological activity feed from `listTenders` and `listDocuments`, deriving four kinds of synthetic events (tender uploaded, tender finalized, document added, document indexed) since there is no dedicated activity-log endpoint yet; the file's own comment notes this can be swapped for a real server-driven feed later without changing how a row renders, because the row-rendering code only depends on the small `Entry` shape assembled here, not on where that data came from.

SettingsPage.tsx is a short, honest settings page: a working theme selector (`light/dark/system`, wired directly to `themeStore`), an account summary block, and an about/build-info block. A code comment states the underlying design principle plainly: a settings page full of switches that do not actually do anything is worse than a short page of only the controls that work,

and features like notification preferences are left out until there is a real backend to back them.

ProfilePage.tsx is currently read-only, since no profile-update endpoint exists on the backend yet, but is a fully designed screen rather than a stub: an account card mirroring the one in Settings, with an avatar built from the user's initials, and an explicit, honest note that editing is coming and that sign-out now lives in the sidebar's account footer rather than the page itself.

Patterns worth reusing

- **Give network code exactly one entry point.** Every fetch in this app funnels through `ApiClient.ts`. That is the only file that knows what an Authorization header looks like, how a 401 gets retried, and how FastAPI's several error-body shapes get unwrapped into one message. Any full-stack React project benefits from this: it means auth, retry, and error-shape logic only has to be written, tested, and fixed once.
- **Separate the “what can I ask the server” layer from the “how do I fetch” layer.** Services (`documentService.ts`, `tenderService.ts`) know endpoint paths and request/response shapes; `ApiClient.ts` knows how to make an HTTP request. Neither knows about the other's concerns. This split is what makes it possible to change how requests are made (say, adding request deduplication) without touching a single page component.
- **Model the wire contract in code, by hand, even when it is tedious.** The files in `models/` are not glamorous, but a renamed backend field becomes a compile error here instead of a silent bug discovered by a user. Writing out keyof-constrained field lists like `toUploadFormData()` does is a small extra step that turns a whole category of “the form sends the wrong field name” bugs into something the compiler catches before the code ever runs.
- **Prefer `as const` arrays plus derived union types over TypeScript `enum`.** This codebase's `tsconfig` sets `erasableSyntaxOnly`, which forbids real TypeScript enums because, unlike almost every other TypeScript construct, an enum actually emits runtime JavaScript code that cannot be stripped by a type-erasure-only transpile step. The `USER_ROLES = ['admin', 'user'] as const` plus type `UserRole = typeof USER_ROLES[number]` pattern used throughout `models/` gets the same compile-time safety, is erasable, and additionally gives you a real runtime array you can iterate over (to build a `<select>`'s options, for instance) for free.
- **Break a circular dependency with a small injection seam, instead of merging the two modules together.** `ApiClient.ts` needs to read the current access token; `authStore.ts` needs to make API calls. Rather than combining them into one large file, `ApiClient.ts` exposes `setAuthProvider()` and `setUnauthorizedHandler()`, and `authStore.ts` calls those once at its own module load. Each file stays focused on one responsibility and the dependency direction is explicit and one-directional at runtime, even though logically the two do depend on each other.
- **Design custom hooks around a state machine you can name, not just a loading boolean.** `useAsyncData` does not just track `isLoading`; it distinguishes a genuine first load from a background refresh, and it keeps stale data visible across a failed refresh rather than clobbering a working screen. `useJobProgress` and `useTenderProgress` both model an explicit `idle / live / polling / offline / closed` state rather than a single connected boolean. Naming the actual states a piece of async UI can be in, up front, tends to produce a hook that handles the edge cases (a slow network, a dropped socket, a cancelled navigation) correctly, rather than discovering them one bug report at a time.

- **A short-lived, single-use ticket is the standard way to authenticate a WebSocket.** Browsers cannot attach custom headers to a WebSocket handshake, so this app authenticates its normal REST calls with a bearer token but authenticates each WebSocket connection with a ticket minted, over an already-authenticated HTTP call, specifically for that one connection and expiring quickly. This pattern generalizes to any browser app that needs an authenticated long-lived connection (WebSocket or Server-Sent Events).
- **Binary, authenticated resources need the object-URL pattern, not a plain `src` or `href`.** Nothing in the browser lets a plain `<img src>`, `<a href>`, or a third-party library's `file` prop attach an `Authorization` header. This codebase's consistent answer is: fetch the resource through the authenticated API client as a blob, turn it into a `URL.createObjectURL()` object URL, hand that object URL to the `<img>` or `<a>` or `react-pdf`, and call `URL.revokeObjectURL()` when it is no longer needed to release the memory. Any app serving private images, PDFs, or downloads behind auth needs this same pattern.
- **A settings screen, or any screen, should not contain a control that does nothing.** `SettingsPage.tsx` states this outright: every toggle on it actually works, and features without a backend yet are left off the page entirely rather than shipped as inert placeholders. This is a small discipline that is easy to skip under deadline pressure and worth deliberately holding onto.
- **Reach for a native browser element before reaching for a library.** `ConfirmDialog.tsx` and `ReassignDialog.tsx` both use the plain `<dialog>` element with its `.showModal()` method rather than a modal library or a hand-rolled focus-trap implementation, and get a focus trap, Escape-to-close, an inert background, and top-layer stacking correct, for free, from the browser itself. The same instinct shows up in `icons.tsx` (hand-drawn SVGs instead of an icon library dependency) and in the choice to keep routing on plain `<Routes>/<Route>` rather than adopting a heavier data-router pattern the app does not actually need. Fewer dependencies means fewer version upgrades to manage and fewer surprises from a library's own internal behavior.

React Project Structure and Workflow, Taught Through VR-Nexus

This document has a narrower job than a file-by-file catalog. It is about the *shape* of a real React project: why the folders are the ones they are, how a request actually moves through them from a click to a rendered pixel, and what decisions you should copy into your own future projects because they generalize well beyond this one app. The worked example throughout is the VR-Nexus frontend: React 19, Vite, TypeScript, React Router 7 (used in its classic `<Routes>/<Route>` form, not the newer data-router API), and Tailwind CSS v4. Every claim below was checked against the actual source files at `myapp/src`, not assumed from convention.

If you have already read `FRONTEND_EXPLAINED.md`, treat this as the companion piece that answers a different question. That document tells you what each file does. This one tells you why the folders exist as boundaries, how data actually flows across them on a real page load, and what to steal for your own project.

1. The folder tree

```
src/
├── main.tsx          # entry point: applies saved theme, starts auth bootstrap, mounts <App/>
├── App.tsx           # trivial root: wraps AppRouter in AppProviders, nothing else
├── index.css         # Tailwind v4 @theme tokens (colors, fonts) and the dark-
mode variant
├── vite-env.d.ts     # ambient types for import.meta.env (VITE_API_BASE_URL)
├── app/              # wiring that is app-wide, not feature-specific
│   ├── router.tsx    # the one route table: every URL in the app, in one file
│   ├── providers.tsx # app-wide context providers (today: just <BrowserRouter>)
│   └── guards.tsx    # RequireAuth / RedirectIfAuthenticated route guards
├── constants/
│   └── routes.ts     # ROUTES object plus URL-pattern builders (tenderDetailPath, etc.)
├── Layout/           # shells that wrap pages: chrome that is rendered once per section
│   ├── AuthLayout.tsx # split-panel shell for login/register/forgot/reset
│   └── DashboardLayout.tsx # sidebar + header + footer shell for the whole signed-
in app
│   ├── DocumentsLayout.tsx # heading + tab bar shared by the three Documents screens
│   └── TenderAnalysisLayout.tsx # heading + tab bar shared by the three Tender Analysis screens
├── pages/            # one component per route, the thing the router actually renders
│   ├── LoginPage.tsx, RegisterPage.tsx, ForgotPasswordPage.tsx, ResetPasswordPage.tsx
│   ├── DashboardPage.tsx, ActivityPage.tsx, SettingsPage.tsx, ProfilePage.tsx
│   ├── DocumentsOverviewPage.tsx, UploadDocumentsPage.tsx, ProcessingPage.tsx, PdfViewerPage.tsx
│   ├── TenderOverviewPage.tsx, UploadTenderPage.tsx, TenderProcessingPage.tsx,
│   ├── TenderReviewPage.tsx, TenderToolsPage.tsx
│   └── AiAssistantPage.tsx, AboutPage.tsx, HomePage.tsx
├── components/       # everything a page renders that is not the page itself
│   ├── dashboard/    # widgets specific to the home dashboard and the app chrome
│   │   (Header, Sidebar, Footer, Panel, StatCard, AnalysisOverview, StorageMeter, ...)
│   ├── documents/    # widgets specific to the evidence-library feature
│   │   (DocumentDropzone, ProcessingTimeline, JobStagePill, TrainingStatusPill, FileGlyph)
│   └── tender/       # widgets specific to the tender-analysis feature
```



```

    (TenderDropzone, TenderTimeline, RequirementReviewCard, ComplianceMatrix,
      EvaluationSimulator, RiskScanner, SubmissionChecker, ReassignDialog, ...)
    └─ feedback/                # generic, feature-agnostic status and error display
      (AlertMessage, AuthAlert, DataState: LoadingRows/ErrorBlock/StaleDataNotice/AsyncSection)
    └─ ui/                      # the shared component kit: no feature knowledge at all
      (ActionButton, Button, TextField, Checkbox, DataTable, ConfirmDialog, icons.tsx, ...)

  └─ hooks/                   # reusable stateful logic, shared across pages
    └─ useAsyncData.ts        # generic "fetch, track loading/error, allow refetch" hook
    └─ useJobProgress.ts      # WebSocket-plus-polling hook for library indexing jobs
    └─ useTenderProgress.ts   # WebSocket-plus-polling hook for the tender pipeline

  └─ services/               # the only code that knows backend endpoint paths and shapes
    └─ authService.ts        # /api/auth/* : login, register, refresh, me, password reset
    └─ documentService.ts    # /api/documents/* and the library indexing jobs
    └─ tenderService.ts      # /api/tenders/* : upload, list, review, finalize, ws-
ticket

  └─ lib/                   # low-level, feature-agnostic utility code
    └─ apiClient.ts          # the ONE place fetch() is called; auth headers, retry, errors
    └─ formatting.ts         # date/number/byte display helpers
    └─ theme.ts              # light/dark/system theme resolution and persistence
    └─ validation.ts         # client-side form validation functions
    └─ pdfWorker.ts          # one-time react-pdf/pdf.js worker setup

  └─ store/                 # cross-cutting client state, readable from outside React
    └─ authStore.ts          # zustand store: signed-in user, tokens, auth actions
    └─ authStorage.ts        # hand-rolled localStorage/sessionStorage persistence
    └─ themeStore.ts         # zustand store: theme preference and resolved theme

  └─ models/                # TypeScript types describing the backend's data shapes
    └─ auth.ts               # User, LoginRequest, TokenResponse, AuthFailure, ...
    └─ dashboard.ts          # UI-only aggregate types the dashboard assembles client-
side

  └─ documents.ts           # library document, job, and stage types + helpers
    └─ tenders.ts           # tender, requirement, evidence-
match types + helpers
    └─ index.ts             # barrel re-exporting auth.ts only

```

2. What each folder is for, and why it stays separate

app/ holds the wiring that makes the app an app: the route table, the top-level providers, the route guards. Nothing here knows about tenders or documents specifically. It holds exactly three things: where every URL goes (router.tsx), what wraps the whole tree (providers.tsx), and who is allowed to see what (guards.tsx). No page-specific logic belongs here; router.tsx's own comments call itself "the one place a route is declared." A route table you cannot read in one sitting is one nobody trusts, and once page logic leaks into it you lose the single file that shows, at a glance, which URLs exist and which are protected.

Layout/ holds shared chrome, the shell a group of pages sits inside, never a page's own content. DashboardLayout renders the sidebar, header, and footer once and gives every signed-in page an <Outlet/>; DocumentsLayout and TenderAnalysisLayout do the same one level down, each owning a heading and tab bar for its own three-screen section. Anything a single page needs and no other page shares belongs in components/ instead. A layout that starts

absorbing page-specific logic stops being reusable, and chrome ends up duplicated anyway to avoid fighting a layout grown too many special cases.

app/ and Layout/ together are why router.tsx stays small: the router only ever nests a page inside a layout and never re-implements a layout's own markup, the whole payoff of nested layout routes covered in section 4.

components/, split into dashboard/, documents/, feedback/, tender/, and ui/, is the clearest lesson here about scaling a component tree. dashboard/, documents/, and tender/ are feature-grouped: everything in tender/ (TenderDropzone, ComplianceMatrix, RequirementReviewCard, RiskScanner) exists because of the tender-analysis feature and would be deleted wholesale if that feature were removed. ui/ is the opposite: ActionButton, TextField, DataTable, icons.tsx know nothing about tenders or documents; they are a small internal kit any feature can reach for. feedback/ sits between the two: AlertMessage and the DataState trio (LoadingRows, ErrorBlock, StaleDataNotice) are generic, but grouped separately because they solve the recurring “show the state of an async operation” problem, distinct from pure input controls. The rule that keeps this from decaying: before adding a component, ask whether it is reusable with no feature knowledge (ui/), reusable for the same cross-cutting problem everywhere (feedback/), or specific to one feature (a named subfolder). Get it wrong and the cost is real: a TenderDropzone in ui/ invites reuse of something that secretly assumes tender-shaped data; an ActionButton buried in tender/ gets reinvented by the next feature because nobody thought to look there.

constants/ holds pure, static data, chiefly ROUTES, the single object of every URL string plus pattern-and-builder pairs (TENDER_DETAIL_PATTERN and tenderDetailPath(id)) for routes carrying an ID. Nothing computed or runtime-dependent belongs here. Both the router and ordinary pages reference the same path strings, and a shared, dependency-free module is what guarantees a `<Link to="/tender-analysis/upload">` typed by hand can never drift from the `<Route path="upload">` in the router; both build from ROUTES.tenderUpload instead.

hooks/ holds reusable stateful logic not tied to one page: useAsyncData, the general-purpose data-loading hook nearly every page uses, and useJobProgress/useTenderProgress, the WebSocket-plus-polling hooks. It holds logic that calls React's built-in hooks internally and packages up behavior more than one component needs, not one-off state a single component owns privately (plain useState), and not direct backend calls that bypass services/. A hook is where “loading, error, refetch” or “socket, poll, reconnect” gets solved once, correctly, under real edge cases (a slow network, a cancelled navigation, a dropped socket); solved inline in every page, it gets solved, and slightly mis-solved, five separate times.

lib/ holds low-level, feature-agnostic utilities: apiClient.ts (the only file that calls fetch()), formatting.ts, theme.ts, validation.ts, pdfWorker.ts. Code here has no opinion about tenders, documents, or any specific screen; a lib/ function could be lifted into an unrelated project unchanged. It knows no backend endpoint path (that is services/) and reads no application state like the signed-in user (that is store/). This is what lets apiClient.ts stay genuinely reusable: it does not import authStore.ts directly (a circular import, since the store also calls the API), so the store instead hands the client two functions, setAuthTokenProvider and setUnauthorizedHandler, at the store's own module load time. lib/ stays clean of app-specific knowledge, and the dependency runs one way at runtime even though the two logically depend on each other.

models/ holds TypeScript types and small pure normalizing functions describing exactly what crosses the wire to and from the FastAPI backend: User, LoginRequest, TenderDetail, Requirement, EvidenceMatch, derived union types like TenderStatus, and helpers like progressFromDetail(). These files render nothing and call no network function. The payoff is concrete: services/tenderService.ts imports its return types from models/tenders.ts, so a backend field rename fails to typecheck immediately, in your editor, rather than surfacing as a silent undefined a user finds weeks later.

pages/ holds one component per route, the thing `router.tsx` actually mounts. A page's job is composition: call a hook or two for data, hand that data to components/, wire up handlers. A large, reusable visual building block used by more than one screen belongs in components/, and raw fetching belongs in services/, not here. A page that grows its own fetch logic, its own child component definitions, and its own validation becomes unreadable as a unit; a page that is mostly composition (call `useAsyncData`, render an `AsyncSection`, render a list of `RequirementReviewCard`) stays legible even at hundreds of lines, because each delegated piece is independently understandable.

services/ holds the only code that knows an actual backend URL path and request/response shape for one resource: `authService.ts` knows `/api/auth/login`, `tenderService.ts` knows `/api/tenders`. It holds thin, named async functions per operation (`uploadTender`, `getTender`, `finalizeTender`), not the mechanics of making an HTTP request (`lib/apiClient.ts`'s job) and no UI state. This split, services knowing "what can I ask the server" and `apiClient.ts` knowing "how do I make a request," lets you change how requests are made (deduplication, retry policy, transport) in exactly one file, touching no page or service function.

store/ holds cross-cutting client state many unrelated parts of the tree need to read or change, readable outside a component's own render cycle: `authStore.ts` (signed-in user, tokens, submit/loading flags) and `themeStore.ts` (theme preference). It holds state that would otherwise force prop-drilling through layers that do not themselves care, or that a non-component module like `apiClient.ts` needs via `getState()`; state private to one component and its children stays as local `useState`. A store that accumulates everything "just in case" becomes a second, competing source of truth harder to reason about than local state, while no store at all forces awkward prop-passing the moment two distant components need the same value.

3. How a request flows end to end

3a. A user logs in

1. The browser requests `/login`. `src/app/router.tsx` matches `ROUTES.login` and renders `<RedirectIfAuthenticated><LoginPage></RedirectIfAuthenticated>`, outside the `RequireAuth`-wrapped `DashboardLayout` block, so no sidebar or header exists yet; `LoginPage` wraps its own content in `AuthLayout`, the split-panel shell shared by all four auth screens.
2. `LoginPage.tsx` reads individual fields from the auth store: `useAuthStore((state) => state.login)`, `.isSubmitting`, `.failure`, `.notice`. Form values (email, password, remember) live in local `useState`, private to this one screen.
3. The user types and submits. `handleSubmit` runs client-side checks (`validateEmail`, `validatePasswordPresent` from `lib/validation.ts`) and, if they pass, calls `login(values)`, the login action from `useAuthStore`.
4. `authStore.ts`'s login action sets `isSubmitting: true`, then calls `authService.login(toLoginRequest(v`, `authService.ts` posts to `AUTH_ENDPOINTS.login (/api/auth/login)` through `api.postJson`, with `anonymous: true` so no stale token rides along on a request that is, by definition, from someone not yet signed in.
5. `api.postJson` lives in `lib/apiClient.ts`, the single file that calls `fetch()`. It sends the request to `${API_BASE_URL}/api/auth/login` (in development, `API_BASE_URL` is empty, so this is a relative path Vite's dev proxy forwards to the backend at `127.0.0.1:8000`), and returns parsed JSON or throws a typed `ApiError`.
6. On success, `authService.login` runs the response through `normalizeUser()` before returning a `TokenResponse` (user, access_token, refresh_token) to the store.
7. `authStore.ts`'s login action calls `writeStoredSession()` (`store/authStorage.ts`) to persist the session to `localStorage` or `sessionStorage` depending on "remember me," then

calls Zustand's `set()` with the new user, `accessToken`, `refreshToken`, `isSubmitting: false`. This `set()` is the moment every subscribed component re-renders.

8. `login()` resolves `true`. `handleSubmit` calls `navigate(readReturnPath(location.state), { replace: true })`. `readReturnPath(app/guards.tsx)` reads the from path `RequireAuth` stashed before redirecting the unauthenticated visitor to `/login`, refusing anything that does not start with a single `/` (closing an open-redirect hole), so the user lands back where they were headed, defaulting to `ROUTES.home`.
9. React Router re-evaluates `app/router.tsx`. The target route sits inside `<RequireAuth><DashboardLayout/></RequireAuth>`. `RequireAuth` reads `selectIsAuthenticated(state)`, now `true`, and renders `DashboardLayout` with the matched page filling its `<Outlet/>`.
10. From here, every authenticated request reads the current token through the getter `authStore.ts` registered via `apiClient.setAuthTokenProvider(() => useAuthStore.getState().accessToken)` at module load, attached as `Authorization: Bearer <token>` inside `request()`. If a request comes back 401 (the access token expires after 24 hours), `apiClient.ts` calls the handler registered via `setUnauthorizedHandler`, `refreshSession()`, which trades the refresh token for a new pair and transparently replays the original request once. No page or service ever thinks about this.

3b. A user uploads a tender and watches live progress

1. The user navigates to `/tender-analysis/upload` (`ROUTES.tenderUpload`), nested inside `<Route path={ROUTES.tenderAnalysis} element={<TenderAnalysisLayout/>}>`, so `TenderAnalysisLayout` renders once and `UploadTenderPage` fills its `<Outlet/>`.
2. `UploadTenderPage.tsx` renders `TenderDropzone` (`components/tender/`), a single-file dropzone restricted to PDF via `TENDER_DROPZONE_ACCEPT` in `models/tenders.ts`. Dropping a file calls `handleFileAccepted`, storing the `File` in local `useState`; nothing is sent yet.
3. The user optionally fills details (reference, authority, deadline, kept as `TenderUploadMetadata` in local state), then presses "Upload & analyse." `handleSubmit` first calls `uploadTender(file)` (`services/tenderService.ts`), which builds a `FormData` and posts via `api.postForm(BASE, body)` (`BASE` is `/api/tenders`), deliberately omitting `Content-Type` so the browser sets the multipart boundary itself.
4. The backend stores the file, creates the tender row, and enqueues the analysis pipeline in one call, returning the new id in milliseconds. If metadata was typed, a second call, `updateTender(id, patch)`, PATCHes it via `api.patchJson`; a failure here does not undo the upload, it just carries a warning forward.
5. `handleSubmit` calls `navigate(\ROUTES.tenderProcessing?tender={id}', ...)`, sending the user to `/tender-analysis/processing?tender=`.
6. `TenderProcessingPage.tsx` mounts, calling `useAsyncData((signal) => listTenders({ limit: 100 }, { signal }), [])` to read every currently-live tender (a bounded window, not a dedicated queue endpoint), and reads `?tender=` via `useSearchParams` to pick which one is focused; an unmatched id falls back to the first row rather than erroring.
7. The focused tender is handed to a `TenderWatcher` subcomponent, where `useTenderProgress(tender.id, { initialProgress: seed })` is actually called (`hooks/useTenderProgress.ts`). Seeded from the list row's last known status, the panel renders something real the instant it mounts, before any connection exists.
8. Inside `useTenderProgress`, an effect keyed on `tenderId` first calls `mintTenderWsTicket(tenderId)`, an ordinary authenticated `POST /api/tenders/{id}/ws-ticket` returning a short-lived, single-use ticket. This exists because a browser cannot attach

an Authorization header to a WebSocket handshake, so the ticket travels in the socket's query string instead.

9. With the ticket, the hook opens new `WebSocket(tenderProgressUrl(tenderId, ticket))`, built via `websocketUrl()` in `apiClient.ts` as `ws://` or `wss://` against the page's own origin; in development, Vite proxies `/ws` to `ws://127.0.0.1:8000`, exactly mirroring the `/api` proxy, so the whole app talks to one origin regardless of transport.
10. As the pipeline advances, the backend pushes frames down the socket. `ws.onmessage` parses each frame, converts it with `progressFromFrame()` into a normalized `TenderProgress`, and calls `setProgress(next)`. That state update re-renders `TenderWatcher`, which passes fresh status, percent, and message into `TenderTimeline`, so the stage rail and status text update live, no polling, no reload.
11. If the socket cannot be sustained, `ws.onerror/ws.onclose` calls `startPolling()`, which calls `getTender(tenderId)` every two seconds and feeds the same `record()` function, so the UI degrades to polling without `TenderWatcher` knowing which transport is active; it only sees progress, failedAt, and connection ('idle' | 'live' | 'polling' | 'offline' | 'closed').
12. When the tender reaches a socket-terminal status (finalized or failed; `ready_for_review` is deliberately not terminal, since the socket stays open through human review), the hook sets connection to 'closed' and an effect in `TenderWatcher` calls `onFinished`, which is `tenders.refetch`, so the queue list beside the focused panel refreshes.

4. How routing and layouts nest

`src/app/router.tsx` is the single source of truth for every URL, and it is worth reading start to finish once, because the whole nesting story is visible in one file. The four unauthenticated routes (`/login`, `/register`, `/forgot-password`, `/reset-password`) are declared first, flat, with no shared layout route wrapping them (each page instead wraps itself in `AuthLayout` individually, since the two forms need different column widths). Everything else hangs off one pathless route:

```
<Route element={<RequireAuth><DashboardLayout/></RequireAuth>}>
  <Route path={ROUTES.home} element={<DashboardPage/>} />
  <Route path={ROUTES.tenderAnalysis} element={<TenderAnalysisLayout/>}>
    <Route index element={<TenderOverviewPage/>} />
    <Route path="upload" element={<UploadTenderPage/>} />
    <Route path="processing" element={<TenderProcessingPage/>} />
  </Route>
  <Route path={TENDER_DETAIL_PATTERN} element={<TenderReviewPage/>} />
  <Route path={TENDER_TOOLS_PATTERN} element={<TenderToolsPage/>} />
  <Route path={ROUTES.documents} element={<DocumentsLayout/>}>
    <Route index element={<DocumentsOverviewPage/>} />
    <Route path="upload" element={<UploadDocumentsPage/>} />
    <Route path="processing" element={<ProcessingPage/>} />
  </Route>
  <Route path={DOCUMENT_VIEWER_PATTERN} element={<PdfViewerPage/>} />
  <Route path={ROUTES.assistant} element={<AiAssistantPage/>} />
  ... (activity, settings, profile)
</Route>
<Route path="*" element={<Navigate to={ROUTES.login} replace />} />
```

This is two levels of layout nesting. The outer route, with no path of its own, has `RequireAuth` wrapping `DashboardLayout` as its element; because a parent route with no path still needs to render, and every child route below it renders inside `DashboardLayout`'s own `<Outlet/>`.

That is what a layout route buys you: `DashboardLayout` (sidebar, header, footer) is written and mounted exactly once, and it survives every navigation between the pages nested under it, rather than being torn down and rebuilt on each link. Compare that to the alternative of every page importing and rendering its own copy of `<Sidebar/>` and `<Header/>`: not only is that more code, it also means the sidebar’s own state (whether it is collapsed, whether the mobile drawer is open) would either have to live in a store just to survive a navigation, or reset itself unnecessarily on every single page change.

One level deeper, `TenderAnalysisLayout` and `DocumentsLayout` repeat the same trick for their own three-tab sections: each is mounted once at `/tender-analysis` or `/documents`, draws its own heading and segmented tab control exactly once, and its three children (index, upload, processing) fill its `<Outlet/>` in turn. The index route (rather than a repeated `/tender-analysis` path on the child) is what lets the parent own the URL outright, so the child route can never register a conflicting path for the same location.

Notice what is deliberately *not* nested a third level down: the tender detail page (`TENDER_DETAIL_PATTERN = '/tender-analysis/:tenderId'`) and the tender tools page (`TENDER_TOOLS_PATTERN`) sit as flat siblings directly under `DashboardLayout`, not as a fourth tab inside `TenderAnalysisLayout`. The router’s own comments explain the reasoning: nesting them as a tab would visually suggest you can tab away from a specific tender’s review screen and tab back to it as if it were a peer of “Overview” and “Upload,” when the real navigation model is “open one tender, then go back to the list.” The same logic keeps the PDF viewer (`DOCUMENT_VIEWER_PATTERN`) out of `DocumentsLayout`’s tabs. This is the general lesson: nesting should mirror the actual navigation model a user experiences, not just the file’s location on disk; a route that behaves like “one specific item opened for reading” belongs as a sibling with a back link, not as a peer of a section’s real tabs.

`app/guards.tsx` is what makes `RequireAuth` actually enforce authentication before that shared layout renders at all: it reads `selectIsAuthenticated` off `authStore`, and if that is false, it renders a `<Navigate to={ROUTES.login} .../>` instead of its children, so `DashboardLayout` and everything nested inside it never mount for a signed-out visitor. Because the guard sits at the layout route’s element, and not repeated inside each individual page, a new page added anywhere under that route inherits the protection automatically; there is no page that can forget to check whether someone is signed in, because the check never lived in the page to begin with.

5. State management map

VR-Nexus uses four distinct places for state, and the choice of which one to use for a new piece of state is not arbitrary; it follows from one question: who else needs to read this, and does it need to survive outside a component’s own lifecycle?

Local component state (`useState`) is for anything private to one screen and its own children: the text typed into `LoginPage`’s form fields, whether `UploadTenderPage`’s details panel is expanded, which row a confirm dialog is currently asking about. Nothing outside that component tree ever reads it, and it disappears the moment the component unmounts, which is exactly correct for a half-typed form field nobody else in the app should see.

Custom hooks (`useAsyncData`, `useJobProgress`, `useTenderProgress`) are for state that is genuinely local to one component’s use of it, but whose *logic* (not the data itself) needs to be identical everywhere it is used. `useAsyncData` is called separately by every page that loads data; each call produces its own independent status/data/error state, but the rules for that state (an `AbortController` per request, keeping stale data visible during a background refresh, telling a true first load from a refetch) are written once and reused. The same applies to the two progress hooks: each call to `useTenderProgress` opens its own socket and owns its own progress/connection state, but the state machine those values move through, and every

edge case around a dropped connection, is defined a single time.

The zustand stores in store/ are for state that many unrelated components across the tree need to read or write, or that a plain module (not a component at all) needs to reach into directly. The signed-in user and their tokens are the clearest case: the sidebar needs the user's name, the API client needs the token on every request, and a login form far away in the tree needs to set all of it at once. `themeStore` is smaller but the same shape: the theme toggle button and the very first paint of `<html>` both need the current theme. The test for “does this belong in a store” is whether passing it down as props would mean threading it through components that do not themselves use it, purely to hand it to a descendant that does, or whether something outside React's component tree (`apiClient.ts`) needs to read it via `getState()`.

Server state fetched fresh via services/ is, deliberately, not cached or centrally stored at all in this app. There is no query-caching library; every page that needs tender or document data calls a service function through `useAsyncData` on its own, and gets a fresh answer. This is a real design choice, not an oversight: `app/providers.tsx`'s own comment marks itself as the future home for a shared query client if one is ever needed, which tells you the team considered it and decided the app is not yet big enough, with enough duplicated fetches across screens, to justify the added complexity. That is the right default for a beginner's own project too: reach for a caching layer when you can point at actual duplicated network calls it would remove, not before.

The reasoning a beginner should carry forward: default to `useState`. Reach for a custom hook only once you notice you are about to write the *same* loading-or-socket logic in a second component. Reach for a store only when a value genuinely needs to be read from two places that are not parent and child, or from outside React entirely. And do not build a caching layer for server data until duplicated fetching is an actual, observed problem, not a hypothetical one.

6. Build and dev tooling, briefly

`vite.config.ts` registers the React and Tailwind v4 plugins, sets the `@` import alias to `./src`, and configures a dev-server proxy: any request to `/api/*` is forwarded to `http://127.0.0.1:8000`, and any request to `/ws/*` is forwarded the same way with `ws: true` set so WebSocket upgrades are proxied too. This one setting is what makes the whole “authenticated WebSocket, same-origin API” story in section 3b work without any special-casing in application code: the browser only ever talks to `localhost:5173` (the Vite dev server), for both REST calls and WebSocket connections, so there is no cross-origin request and no CORS configuration needed on the FastAPI side at all, and `websocketUrl()` in `apiClient.ts` can build a relative `/ws/...` URL against the page's own origin and trust the proxy to route it correctly. The config's own comment notes it binds to `127.0.0.1` rather than `localhost` as the proxy target specifically because Node can resolve `localhost` to the IPv6 loopback address, which fails if the backend is only listening on IPv4. Beyond that, TypeScript's project references (`tsconfig.app.json` for the app code, `tsconfig.node.json` for Vite's own config file) exist mainly so the `@` path alias and compiler options can differ between “code that runs in the browser” and “code that runs in Node while building,” and `npm run typecheck` runs both before every build; this is standard Vite-plus-TypeScript scaffolding and is not worth digging into further as a beginner.

7. Checklist for starting a new React project from scratch

1. **Pick a routing library and decide the layout-route pattern up front.** Classic `<Routes>/<Route>` or a data-router, decide before ten pages exist whether shared chrome

(nav, sidebar, tabs) lives in nested layout routes with an `<Outlet/>`, so you never copy-paste a header into every page.

2. **Give network code exactly one entry point.** One thin module (`lib/apiClient.ts` here) is the only place `fetch()` is called, so auth headers, retries, and error-shape parsing exist in one file, not reinvented per request.
3. **Separate “what can I ask the server” from “how do I make a request.”** Named, typed functions per backend resource in a `services/` layer that calls your API client, not `fetch()` from components, so request mechanics can change later without touching a single page.
4. **Model the wire contract with real types, colocated by resource.** TypeScript types for exactly what your backend sends and expects, one `models/` file per resource, so a renamed backend field is a compile error, not a silent runtime bug a user finds first.
5. **Group components by feature, and keep a separate feature-agnostic UI kit.** Decide early what is reusable because it knows nothing about your domain (buttons, inputs, tables) versus reusable because it solves one feature’s problem, and fold them into different folders so a contributor can tell at a glance what is safe to reuse anywhere.
6. **Reach for a custom hook once loading or async logic repeats, not before.** The first page can call a service function inside a plain `useEffect`. The second page needing the same loading/error/refetch behavior is the signal to extract a hook like `useAsyncData`, solving cancellation and stale-data handling once.
7. **Put a dedicated store layer between “local state” and “the whole app.”** A small state library only for values multiple unrelated components need, or that non-component code must read directly; everything else stays local `useState` rather than defaulting to a global store.
8. **Solve WebSocket auth with a short-lived ticket, not the access token in the URL.** Browsers cannot attach custom headers to a WebSocket handshake, so mint a one-time, quickly expiring ticket over an authenticated HTTP call right before opening the connection.
9. **Wrap authenticated binary resources in an object-URL fetch, never a bare `src` or `href`.** A plain `<img src>` or `<a href>` sends no Authorization header; fetch as a blob through your authenticated client, turn it into a `URL.createObjectURL()`, and revoke it when done.
10. **Configure your dev server’s proxy before writing any fetch code.** Route `/api` (and `/ws`, for WebSockets) to your backend, so the browser only ever talks to one origin in development and CORS never comes up.
11. **Keep your true application root trivial.** A root that only wraps a router in your top-level providers means there is always exactly one obvious place to add the next app-wide concern.
12. **Decide folder boundaries by asking “what does NOT belong here,” not just “what does.”** Every folder needs a one-sentence responsibility and a clear misfit answer, written down before it fills up, because a boundary nobody can articulate erodes the first time someone is in a hurry.

VR-Nexus Viva Preparation: Questions and Answers

A study aid for defending VR-Nexus, a Tender Intelligence and Evidence Matching Platform, in a viva, interview, or demonstration. Organized by topic, each with Q/A pairs written the way a student would say them out loud. It draws only on what the codebase and its own documentation show, including the parts that are unfinished, simplified, or dead code, because “what would you do differently” tests honesty as much as competence.

1. Project Overview and Motivation

Q: In one sentence, what does VR-Nexus do? A: It takes an uploaded procurement tender PDF, often hundreds of pages, automatically extracts every requirement it contains, matches each requirement against a company’s own evidence library using semantic search, and produces a reviewable Excel tracker plus a downloadable output package a bid team can submit.

Q: Who is the user, and what does their day look like without this tool? A: A bid or proposal team member, typically non-technical, at a company responding to tenders. Without automation someone has to read a 200 to 400 page document, manually copy every clause that looks like a requirement into a spreadsheet, judge whether it is mandatory, work out its evaluation weight, then separately dig through case studies, certificates and methodology documents for proof the company can meet it. That is slow, error-prone under deadline pressure, and does not scale across several simultaneous bids.

Q: What is actually automated versus left to a human? A: Reading the document, extracting requirements into a structured form, and finding candidate evidence via vector search are automated. The judgment call is not: every proposed match must be accepted, rejected, or reassigned by a human before a tender can be finalized, and nothing auto-finalizes. Getting a requirement or a match wrong in a real submission has real consequences, so the AI produces a first draft for review, not a final answer.

Q: What does “evidence matching” mean concretely? A: Every library document is chunked and each chunk is embedded into a vector locally. Each extracted requirement’s text is embedded the same way, and the system runs cosine-similarity search against the stored vectors in Postgres via pgvector. A match at 0.85 similarity or above auto-matches; 0.60 to 0.84 is offered as a suggestion a human must confirm; below that is treated as no evidence found and flagged. So it is “search by meaning across the company’s own library, with similarity confidence deciding how much the system trusts its own suggestion.”

Q: Isn’t it risky trusting an LLM to extract requirements accurately? A: Yes, and the design treats it that way. Extraction runs a strict JSON schema per chunk with deduplication, and the pipeline fails hard rather than continuing silently: if over 30 percent of chunks fail after retries, or zero requirements come out, the whole run raises rather than producing an incomplete result that looks finished. Compound or non-standard tender wording (“No/Advisory,” “Financial / Pass-Fail”) is kept verbatim in a raw column alongside the normalized value, so nothing gets silently rewritten. And nothing is finalized without human review.

2. High-Level Architecture

Q: Walk through the major pieces and how they talk to each other. A: The client is a React SPA served by Vite. It talks over HTTP to FastAPI, which reads and writes PostgreSQL (with pgvector) through async SQLAlchemy. Slow work is never done in the request handler:

FastAPI writes a job row and hands it to Celery, using Redis as the broker between the API process and the worker process. The worker does the heavy lifting (parsing, OCR, LLM calls, embeddings) and publishes progress to Redis as it goes. The frontend holds a WebSocket connection to a small FastAPI route that subscribes to those Redis messages and forwards them live. The sixth piece is Anthropic Claude, used for extraction, auto-tagging, and grounded question-answering.

Q: Why do you need a background worker instead of doing everything in the request handler? A: An HTTP request has an implicit deadline; a client, proxy, or load balancer eventually gives up. Extracting 150-plus requirements by calling an LLM once per chunk, or OCRing dozens of scanned pages, can take minutes. Doing that inline means either the request times out with the client unsure whether it succeeded, or a web thread is tied up per upload, which does not scale. Celery splits “describe the work” from “do the work”: the route creates a row and enqueues a task, returning in well under a second, while a separate worker process does the actual job independently of any request’s lifetime.

Q: How does the browser see progress if the request already returned? A: Through a WebSocket, not polling. The worker publishes progress payloads to a Redis Pub/Sub channel and also writes the latest payload to a plain key with a TTL, so a snapshot always exists. The frontend opens a WebSocket that, on connect, immediately sends the current snapshot (so a reconnecting client sees current state right away) and then subscribes to the channel for everything after. The server pushes the instant something changes, instead of the browser asking “done yet?” every second.

Q: Redis plays three roles here. What are they, and why one instance? A: Celery broker (the task queue), progress cache (Pub/Sub plus the snapshot key), and short-lived single-use tokens (WebSocket tickets, password-reset redemption), leaning on Redis’s atomic operations (GETDEL, SET NX) and built-in key expiry. One instance doing three well-separated jobs keeps the operational footprint small for a project this size; splitting them onto separate instances would be reasonable at much larger scale.

Q: What happens if Redis goes down mid-pipeline? A: New tasks cannot be enqueued or picked up, and a running worker loses its ability to publish progress the moment it tries. A task’s in-memory computation might continue briefly, but the broker dependency means this is a real single point of failure with no fallback broker or persistence layer today, worth naming honestly rather than glossing over.

3. Tech Stack Justification

Q: Why FastAPI over Django or Flask? A: Native async support (the app spends most of its time waiting on I/O), automatic request validation and OpenAPI docs from Pydantic type hints, and a dependency-injection system that let authentication and role checks live in one place (`get_current_user`, `require_role`) instead of being reimplemented per route. Django would bring an ORM, admin panel and templating layer the project doesn’t use, and its async story was less mature when this stack was chosen. Flask is closer in spirit but would mean hand-rolling the validation and DI layer FastAPI gives for free.

Q: Why async SQLAlchemy rather than the classic sync ORM? A: The API is I/O-bound, mostly waiting on the database or a network call, not computing. Async SQLAlchemy lets one process handle many concurrent requests without blocking a thread per query, which matters because FastAPI runs on an async event loop and blocking calls would stall it. It costs real complexity (sessions, engines and query patterns all have async-specific quirks), but for an I/O-bound API it’s the right tradeoff.

Q: Why Celery and Redis instead of FastAPI’s own BackgroundTasks? A: Background-

Tasks run in the same process, after the response, fine for something quick but wrong for work that can take minutes and needs to survive the web process restarting, needs retries and time limits, and needs to be observable and cancellable externally. Celery gives a durable, distributed queue: a task queued by the API stays in Redis until a worker, possibly on another machine, picks it up, and settings like `task_acks_late` and time limits give real guarantees about worker crashes. Worth the operational overhead for a pipeline calling an LLM 150-plus times per tender.

Q: Why Postgres with pgvector instead of a dedicated vector database? A: Embeddings are one property of documents that already need to live relationally alongside users, tenders, requirements and audit logs. Keeping vectors in the same Postgres instance means they share transactions, backups, and joins with everything else, a chunk's category and its parent document's metadata can be filtered together with its embedding in one query. `pgvector` adds a native vector type and fast nearest-neighbor operators (`<=>` for cosine distance) directly inside Postgres, avoiding a whole extra system to deploy and keep in sync. A dedicated vector database would likely win at much larger scale, but not at this project's scale.

Q: The plan specified Next.js; the build is Vite. What happened, and is that a problem?

A: A genuine, documented deviation. The Implementation Plan called for Next.js 15 with the App Router; what was actually built is Vite plus React 19 using classic React Router (`<Routes>/<Route>`, not the data-router loaders/actions API). Functionally the two are similar for this app's needs, a single-page authenticated dashboard with no requirement for SSR or file-based routing, and nothing depends on a Next-specific feature. But it's a real deviation, likely drift across a team split across branches, that was reviewed afterward and deliberately kept rather than migrated, with Vite's simpler build model and faster dev server cited as a reasonable, if retroactive, justification.

Q: Why Tailwind, and specifically v4? A: Utility classes styled directly in JSX avoid separate CSS files drifting out of sync across a multi-person project. `v4` specifically runs as a Vite plugin rather than through PostCSS, so there is no separate `tailwind.config.js` or `postcss.config.js`; design tokens (colors, fonts, semantic surface/border tokens) are defined in CSS via the `@theme` directive in `index.css`, letting the same class names resolve differently under a dark-mode selector with no JS-side theme logic.

Q: Why Anthropic Claude, and why two models (Sonnet and Haiku)? A: Claude is the single provider for every AI feature (extraction, auto-tagging, grounded Ask), reached through one API key via the OpenAI-compatible endpoint; the `openai` package here is just HTTP plumbing to Anthropic, not a second provider. The two-model split is cost and latency, not capability: extraction runs 150-plus times per tender doing mechanical, well-specified work ("copy this field verbatim"), so it uses the cheaper, faster Haiku-tier model (`ANTHROPIC_EXTRACTION_MODEL`). Grounded Ask has to weigh several retrieved passages and produce connected, cited prose, which justifies the stronger Sonnet-tier model (`ANTHROPIC_MODEL`). Making the model a per-call-site setting rather than one global constant turns that tradeoff into a config change, not a code change.

Q: Why local embeddings instead of a hosted embeddings API? A: Anthropic doesn't serve an embeddings endpoint at all, so a hosted API would mean a second provider and a second API key purely for that one capability. More generally, a second external dependency is a second point of failure and a second credential to manage. The app runs `fastembed` (`BAAI/bge-base-en-v1.5`, 768-dim, ONNX, CPU-only) inside the Celery worker, no network call. The cost is a one-time model download and CPU time per document, slower than hosted throughput, but it removes an entire "the embeddings API is down or rate-limited" failure mode for the volume this project handles.

4. Security Design Decisions

Q: Explain the access and refresh token design, and why two tokens. A: Login issues a short-lived access token (24 hours) sent on every request, and a longer-lived refresh token (7 days) used only to get a new access token via a dedicated refresh endpoint. The token used constantly is short-lived to limit exposure if it leaks; the refresh token, used rarely, can safely live longer without daily re-login. `decode_token` only checks signature and expiry, not token type, so each caller must check the type claim itself (access route requires `type == "access"`, refresh route requires `type == "refresh"`), stopping a leaked refresh token from working as an access token. Honest gap: refresh is stateless, the old refresh token isn't invalidated when a new pair is issued, so it isn't rotating, and a leaked refresh token stays valid until it naturally expires.

Q: How does account lockout work, and why implement it? A: Login checks `locked_until` before checking the password. A wrong password increments `failed_login_attempts`, and reaching a threshold (five, by default) locks the account for a configured window, slowing brute-force password guessing. Login returns the identical "Incorrect email or password" error whether the email doesn't exist or the password is wrong, so the endpoint can't be used to enumerate registered emails. A successful reset also clears any lockout, since a forgotten password is often the actual reason for the lockout.

Q: Explain the WebSocket ticket system fully, including why GETDEL matters. A: A browser's WebSocket constructor can't set custom headers, so it can't carry `Authorization: Bearer <token>`. Putting the real access token in the connection URL instead would leak a 24-hour credential into browser history, proxy access logs, and potentially a `Referer` header for as long as it stays valid. The fix: an already-authenticated HTTP endpoint mints a random 32-byte ticket and stores it in Redis with a 60-second TTL, scoped to a specific resource, so a ticket for one tender can't be pointed at another. The frontend connects using that ticket instead of the real token. `redeem_ticket` uses Redis's GETDEL, reading and deleting the key as one atomic operation, instead of a separate GET then DELETE. A WebSocket handshake can be retried or double-fired (a React effect re-running, a reconnect racing the original attempt), and a naive read-then-delete leaves a race window where two near-simultaneous attempts could both see the ticket as valid before either deletes it. GETDEL closes that window structurally: only the first caller ever succeeds, the second sees nothing, not a rare bug but something the operation makes impossible.

Q: How is the password reset token made single-use, given a JWT's signature alone can't guarantee that? A: A bare JWT verifies successfully every time it's presented until it expires, no matter how many times. Reset tokens travel through email, where they can sit unread or get clicked twice by accident, so `create_password_reset_token` mints a random `jti` claim, and redemption goes through a Redis `SET ... NX` check keyed on that `jti`: the first successful redemption marks it used, every later attempt with the same token is rejected even though the JWT itself still verifies. `SET NX` performs "is this used, and if not, mark it used" atomically, closing the same kind of race window GETDEL closes, using the mirror-image primitive for "claim once" instead of "read and remove."

Q: Is there per-user ownership on tenders and documents? What about IDOR risk?

A: No, deliberately. The Evidence Library and tender workspace are shared resources for a whole internal team; any signed-in user can view, edit, or delete any tender or document, and `uploaded_by` exists for attribution, not access control. This was flagged explicitly in an external code review as a potential IDOR issue, since neither `_get_tender` nor `_get_document` checks ownership. The decision, made consciously, was to keep it fully shared, because this is a small internal tool for one team, not multi-tenant SaaS. That reasoning is now documented directly in both functions as an explicit trust boundary. It's a defensible answer at this scale, but it would need to change the moment this product serves more than one organization's data in one deployment.

Q: Why do secrets live in .env and never get committed? A: A JWT secret, database credentials, or an API key committed to git is effectively permanently compromised the moment that history is pushed anywhere, deleting it in a later commit doesn't remove it from prior history. Keeping secrets in an untracked .env, read once via a cached settings object instead of scattered os.getenv calls, means values only exist where actually configured, and rotating a compromised secret doesn't require rewriting history. alembic.ini follows the same discipline: sqlalchemy.url is left blank and filled at runtime from settings, so the connection string exists in exactly one place.

5. The Tender Analysis Pipeline End to End

Q: Walk through upload to visible progress. A: The upload route is thin: validate the file (PDF, under 100MB, non-empty), create a Tender row in UPLOADED state, save the bytes under a path keyed by the tender's UUID, publish an initial progress message, and enqueue the Celery task, returning in well under a second. Everything from parsing onward happens in that task in a separate worker process. The frontend mints a WebSocket ticket over an authenticated call, opens a progress socket that replays current state immediately, then streams updates as the worker moves through parsing, chunking, extraction, matching, reporting and folder assembly.

Q: How does chunking and extraction work, and why not send the whole document at once? A: A tender can run hundreds of pages, larger than most context windows can handle reliably, and even where it would fit, extraction accuracy suffers when asked to find everything in one huge pass. The pipeline extracts per-page text and tables via PyMuPDF, falling back to Tesseract OCR on pages whose extracted text is suspiciously short (a signature of a scanned image). It then does section-aware chunking: detecting section boundaries by pattern, grouping pages by section, splitting each into roughly 2000-token windows with roughly 200 tokens of overlap, keeping a running token-to-page map so every chunk carries an accurate page range. Overlap matters because a requirement can fall across a chunk boundary; without it, that requirement could be silently lost. Extraction then runs one LLM call per chunk, bounded by a concurrency limit so 150-plus chunks run several at a time rather than sequentially or all at once against a rate limit. Requirements from overlapping chunks are deduplicated by hashing clause reference plus description.

Q: How is Cancel or Retry mid-pipeline handled safely if a worker might already be running? A: Via a monotonically increasing run_generation counter on the Tender row. Every write the pipeline makes is an atomic conditional update: UPDATE tenders SET ... WHERE id = :id AND run_generation = :expected_generation, never a separate read then write. Cancel or Reprocess immediately bumps run_generation. The next time the running worker tries to write progress, its captured expected_generation no longer matches, the update affects zero rows, and the code treats that as "superseded" and stops cleanly without needing to be told why. expected_generation is captured once at enqueue time, not re-read when the task starts, because a task can sit queued behind other work, and a fresh re-read at start time could wrongly claim ownership of a newer generation bumped during that wait, causing two tasks to race as if they were the same run. Cancel also issues a hard Celery revoke(terminate=True) using the stored celery_task_id, so the generation guard and the hard revoke work together.

Q: What does the failure path look like, separate from cancellation? A: A dedicated _fail path gated by the same run_generation check, because TenderCancelled just means someone acted, and if that action was Stop immediately followed by Retry, a new worker may already be running by the time the old worker's exception handler catches up; writing FAILED unconditionally would clobber the new run's live status. _fail also calls db.rollback() before reading the tender's status, so the failing stage's half-committed work is discarded first,

making the recorded `failed_stage` describe the last stage that actually completed rather than a half-applied leftover.

Q: What does the reviewable output actually contain? A: A four-sheet Excel workbook (Requirements, Summary, Evidence Matches, Instructions) plus a zip with the original tender PDF and a “Required Documents” subfolder of the actual evidence files for every covered requirement. The Requirements sheet reproduces the tender’s own wording verbatim wherever it doesn’t fit a clean enum, normalized values exist only for sorting and coverage math. “Coverage” is computed consistently: a match counts only if a human accepted it, or it auto-matched above 0.85 and hasn’t since been rejected. A narrative clause the tender never asked to be proven (`evidence_required` false) is excluded from “missing evidence” rather than mislabeled. The folder is rebuilt from scratch on every finalize, so a rejected match never leaves a stale file behind.

Q: Can a run fail with zero requirements, and how does that differ from finding nothing legitimately? A: Zero requirements is treated as a hard failure, not a valid empty result, because a run showing 40 of 400 real requirements as “finished,” with nothing telling the user, is worse than one that fails loudly and points at what to check. Without this check a run could reach `ready_for_review` with an empty tracker presented as success, which was an actual observed bug, distinct from genuinely finding no usable text (a scanned page with no text layer).

6. The Evidence Library and RAG Pipeline End to End

Q: Walk through upload to searchable. A: Upload and training are separate. Uploading only validates the file, computes a SHA-256 hash, checks for exact or near duplicates, and stores it with a Document row in QUEUED state, nothing is parsed or embedded yet, so no CPU is spent before a duplicate check might reject it. A separate Train call enqueues indexing, either for specific ids or everything queued; already-indexed documents are skipped unless forced, which is what makes retraining incremental. Indexing is one Celery task per document (so training one new asset never touches the rest of the library) walking Parsing, Tagging (filling metadata the uploader left blank, keyword heuristics first, cheap LLM tier as last resort), and Embedding (chunking and embedding in batches so progress reports incrementally on a large document).

Q: How does parsing differ by document type? A: Case studies resolve into six canonical sections (Client, Sector, Scope, Challenge, Solution, Results) via a heading-synonym table, falling back to an LLM section map if fewer than three sections are found, and to one undifferentiated blob only as a last resort. Company documents (certificates, filings) are deliberately not split into sections at all, since fragmenting a short certificate would divorce “Scope of certification” from the certificate number that gives it meaning; instead the parser classifies type by weighted keywords and extracts identifiers (certificate number, dates) onto the document row. Methodology documents split by phase markers (“Phase 2,” numbered headings), since that content is sequential and a phase label tells a reader where a retrieved passage belongs, something a bare similarity score can’t convey.

Q: How does search work, and what does “grounded” mean for Ask? A: Search runs cosine-distance queries directly against Postgres via pgvector’s `<=>` operator, no in-memory index or cache, so a document indexed a moment ago is immediately searchable. Ask builds on retrieval with a strict rule: the model only sees retrieved chunks, is instructed to answer using only those, and must produce an exact sentinel string when the context doesn’t answer the question. Chunks are numbered in the prompt, and the answer is parsed afterward for those citation numbers; an uncited retrieved chunk is still returned, since knowing what was considered but unused is itself useful. `grounded` is true only when the answer cites at least

one source and doesn't decline, so confident prose with nothing cited is correctly reported as ungrounded.

Q: What happens if the library has nothing relevant, or no API key is configured? A: If retrieved passages don't answer the question, the model declines and the response comes back grounded: `false`, with the retrieved passages still shown rather than hidden, since honestly showing the closest evidence is more useful than showing nothing. With no API key, `complete()` in the LLM wrapper returns an empty string on failure rather than raising, and every call site tolerates that and falls back to heuristics, so the whole library pipeline still runs end to end without machine-assisted tagging or generated answers. The frontend's "Find" mode skips generation entirely and returns ranked passages directly, working with zero LLM configuration.

Q: How does the same library serve both tender matching and Ask? A: Both read the exact same Chunk table and embeddings through the same search module; tender evidence matching calls the same seam the `/search` REST endpoint calls. An answer through Ask and a match surfaced during tender review are always grounded in the same documents, with no separate, potentially drifting copy of "what evidence exists."

7. Database Design

Q: Why Alembic migrations instead of auto-creating tables? A: Auto-creation works for a throwaway prototype but breaks the moment more than one environment needs the same schema, and leaves no record of why a change happened. Alembic keeps a versioned history (`upgrade()`/`downgrade()` per migration) plus a table recording what's applied. A manual ALTER TABLE instead means no other environment picks it up, no record of why, and Alembic's bookkeeping now disagrees with reality, which can break the next real migration or silently do the wrong thing. This project hit the concrete failure mode migrations prevent: several Requirement fields (`responsibility`, `dpl`, `prime`, others) were added to the model but a migration was initially missed, and every tender run failed with `UndefinedColumn` until it was added.

Q: Why is the embedding column vector(768), and why does that matter operationally? A: 768 is the output width of the embedding model in use (BAAI/bge-base-en-v1.5). It isn't a free parameter later, two vector columns are already committed to it; switching models to a different width means a real column migration and re-embedding every previously-indexed chunk, not just relabeling. The embedding provider checks at runtime that returned vectors actually match `EMBEDDING_DIM` and raises immediately with an actionable message, rather than letting a mismatch surface later as an opaque pgvector insert failure inside a Celery task.

Q: What is the "tombstone" pattern, and why does it matter here? A: This backend merged two earlier prototypes, a flat one with no real auth or database, and a fuller tender-analysis system. Where both had a same-named module and only one is live, the losing file wasn't deleted, its body was replaced with a docstring explaining the situation and a `raise ImportError(...)` naming the live replacement. Quietly deleting relies on nobody importing it from memory or an old branch; leaving it unused but importable is actively dangerous, since a stray import could silently run old, incompatible code built against a no-op auth stub. A tombstone fails loudly at the exact line instead of failing mysteriously elsewhere or not failing at all. It shows up at real scale here: `app/config.py`, `app/db.py`, `app/api/library.py`, `app/api/ws.py`, several `app/services/` modules, and a whole tombstoned parser package all follow it.

Q: Is there audit logging, and what does it capture? A: A deliberately generic `AuditLog` table (`action`, `resource_type`, `resource_id`, a free-form JSON details blob) covers logins,

tender runs, document training and user-management actions without a table per action type. `user_id` is nullable, since some events (a failed login against an unknown email, a system-level error) have no authenticated user to attach to.

Q: Why does Requirement have both normalized and raw verbatim fields? A: Real tenders write compound or non-boolean values a clean enum can't represent ("No/Advisory," "Financial / Pass-Fail"). Storing only a normalized value would force the tracker to either drop the tender's actual wording or approximate it, and the tracker's job is to reproduce the client's own wording exactly. So the model keeps both: a normalized column for sorting, filtering and coverage math, and a `_raw` column with the tender's exact text, with output assembly always using the raw fields where they exist.

8. Frontend Architecture

Q: Explain the routing and layout nesting. A: Classic React Router `<Routes>/<Route>` (not the data-router loaders/actions API). Layout routes render a shared shell with an `<Outlet />`: `DashboardLayout` is the outermost authenticated shell (sidebar, header, footer); inside it, `DocumentsLayout` and `TenderAnalysisLayout` each render their own tab bar with a nested outlet; the page component renders inside that. The sidebar and tab bar are written once each, and switching tabs doesn't remount the outer shell. One deliberate exception: a tender or document's detail/review page is not nested as a fourth tab, it sits as a sibling route directly under `DashboardLayout`, to avoid implying you can tab away from a review screen and back, when the real model is "open one item, then go back to the list."

Q: How is state managed, local versus store versus server data? A: Component-private state stays in `useState`; state genuinely shared across the tree, or that must survive a navigation without refetching, goes into a Zustand store. There are exactly two: `authStore` (user, both tokens, bootstrap/submission state) and `themeStore` (light/dark/system preference). Everything else that looks like data is server state, loaded via `useAsyncData` (or `useJobProgress/useTenderProgress` for WebSocket progress), which tracks loading, error and refresh itself and stays with the page that requested it rather than living globally.

Q: Why a thin API client wrapper instead of calling fetch() from components? A: `apiClient.ts` is the only file that calls `fetch()` directly; it attaches the current access token, retries once on 401 using a single-flight pattern (several near-simultaneous 401s trigger only one refresh, the rest await it), and unwraps FastAPI's several error-body shapes into one message. A service layer (`authService.ts`, `documentService.ts`, `tenderService.ts`) sits on top knowing endpoint paths and shapes but nothing about how a fetch happens; components only call service functions. Changing how a request is made never touches a page component; changing an endpoint never touches the low-level fetch logic.

Q: How are the two progress WebSockets consumed on the frontend? A: Two hooks, `useJobProgress` (library indexing) and `useTenderProgress` (tender pipeline), sharing a state machine (idle, live, polling, offline, closed) rather than a boolean. Each first mints a ticket over HTTP, then opens the socket with that ticket instead of the real token. Three consecutive connection failures fall back to polling the REST endpoint every couple of seconds, so a corporate proxy blocking WebSocket upgrades doesn't leave the UI stuck. `useTenderProgress` treats `ready_for_review` as not socket-terminal, since the connection needs to stay open through human review; only finalized/failed close it. Both null out the socket's handlers before calling `.close()`, since closing fires `onclose`, and without nulling first, that teardown event would itself try to start polling right as the component unmounts.

Q: How does the frontend display private, authenticated files like a tender PDF? A: Nothing lets a plain `<img src>` or `<a href>` attach an Authorization header. The consistent pattern: fetch the resource as a blob through the authenticated client, convert it

via `URL.createObjectURL()`, hand that object URL to the `<img>`, `<a>`, or `react-pdf`, and call `URL.revokeObjectURL()` when done. Used for a tender's source PDF and generated output, and a library document's original file and extracted page images.

9. Testing, Reliability, and Failure Modes

Q: What happens if a Celery worker crashes mid-job? A: `task_acks_late=True` means a task is only marked complete after it finishes, so a crash mid-task doesn't lose the message, it can be redelivered. `worker_prefetch_multiplier=1` stops one worker from hoarding several jobs while a sibling sits idle, which matters for this heavy-per-task workload. On the database side, `run_generation` fencing means even overlapping or re-run tasks for the same tender just become no-ops if superseded, rather than corrupting the row. What's not automatic is a mid-failure full retry: library indexing is registered with `max_retries=0`, since a parse failure is almost always a bad file rather than transient, so an automatic retry would just re-run expensive OCR pointlessly; failures are recorded for a human to explicitly re-trigger.

Q: What happens if the LLM call fails after retries? A: In tender extraction, each chunk retries individually, and the pipeline tracks failure rate overall: over 30 percent of chunks failing, or zero requirements extracted, raises a hard `ExtractionFailed` rather than continuing incomplete. Three dedicated failure columns (`failed_stage`, `error_detail`, `failed_at`) exist because `progress_message` gets overwritten by every later stage, so without them there'd be no record of which stage actually died. In the library, the LLM wrapper is tolerant by design: `complete()` returns empty string on failure instead of raising, and every call site falls back to heuristics or an honest "no answer," which is why the library pipeline still functions with no API key configured.

Q: What does the health check endpoint verify, and why does it matter operationally? A: `GET /health` runs a real `SELECT 1` against Postgres, so a liveness probe proves the database is reachable, not just that the process is alive. `GET /api/health/llm` round-trips a trivial prompt through the exact same Anthropic client configuration real extraction uses. Without it, an operator whose extraction is failing has to guess between a bad key, a retired model name, a network block, or a spent quota, usually by re-running a whole tender and reading worker logs; this answers it in seconds. It always returns HTTP 200, distinguishing "the probe ran, here's what it found" from "the probe itself crashed," so a caller checks one field in one response shape.

Q: What test coverage exists, and what is honestly missing? A: Backend unit tests cover storage path-safety, both chunkers, both parsers, metadata precedence, duplicate detection, and RAG failure modes, all without a live database. Missing: any test of the HTTP/auth layer, the Celery pipeline end to end, or true integration tests, all of which need a live Postgres with pgvector that the current suite avoids. Frontend testing got a Vitest setup later with one model-layer test file; before that there was no frontend test framework at all. There's also no load/performance testing and no dedicated security pass beyond fixes from one external code review.

10. Tradeoffs and Honest Limitations

Q: What are the real limitations, honestly?

1. **No per-user access control on tenders or documents.** Deliberate for a small shared team tool, but a real IDOR-shaped gap the moment this needs to serve more than one organization. Identified in review and consciously kept, a legitimate but load-bearing tradeoff.

2. **Refresh tokens aren't rotated or revocable.** A valid refresh token mints new access tokens for its full lifetime; there's no server-side blocklist to invalidate one early (a stolen laptop, say). Production handling more sensitive data would want rotation and revocation.
3. **A fully-built support-email feature is dead code.** `services/support_email.py` composes and sends a failure's full detail over SMTP, well-documented and ready, but nothing live calls it; the real "contact support" flow is a frontend `mailto:` link, with the backend only recording that the hand-off happened. A concrete lesson that a confident module isn't proof it's wired into anything.
4. **Some dead code was never tombstoned.** Most leftover prototype code is deliberately tombstoned, but `app/core/ws_manager.py` is genuinely dead, never converted, just quietly unreferenced after progress broadcasting moved to Redis Pub/Sub. A smaller version of the risk tombstoning exists to prevent.
5. **Duplicated, drifting stage labels.** `pipeline_stages.py` and `progress.py` both define near-identical stage labels with slightly different text, and only `progress.py`'s copy reaches the client. Known and deliberately unfixed, since consolidating them would change strings the frontend may depend on.
6. **Local CPU-only embeddings are a real throughput ceiling.** Avoids a second external dependency, but computation scales with the worker's own CPU, not a hosted service's throughput. A much larger library or upload volume would hit this ceiling.
7. **Real concurrency and integration test gaps.** The most concurrency-sensitive code, `run_generation` fencing, WebSocket ticket redemption, the Celery pipeline end to end, has no automated coverage; what exists is unit-level coverage of parsers, chunkers, and pure logic. A genuine risk area, not just a nice-to-have.

Q: If forced to fix one before a second customer, which and why? A: Per-user access control, without much hesitation. It's the one limitation that isn't a missing nicety but an actual absent security boundary, and it becomes dangerous the moment this serves more than one company's tenders and evidence in one deployment, since right now any signed-in user, from any company sharing that deployment, can see and modify any other's data. The others are quality and maintainability issues, not a data-isolation failure.

11. Future Work: "If You Had More Time"

Q: What would you build next with another month?

1. **Real per-tenant or per-team access control**, extending `uploaded_by` attribution into an actual authorization check scoped by organization or team, preserving the shared-workspace model while letting the platform serve more than one company.
2. **Rotating refresh tokens with server-side revocation**, storing issued tokens (or their `jti`) server-side, invalidating the old one on reissue, and adding a "log out this device" or "revoke all sessions" action.
3. **A real integration suite against live Postgres with pgvector**, covering the HTTP/auth layer and specifically the concurrency-sensitive paths (`run_generation` fencing under simulated Cancel/Retry races, WebSocket ticket redemption), the areas most likely to hide a subtle bug unit tests on pure functions can't catch.
4. **Wire up or remove the orphaned support-email feature**, either actually calling `send_failure_report` from the failure path so an operator gets a real traceback auto-

matically, or removing the module so a future reader isn't misled by a well-documented function that never runs.

5. **A real activity feed and dashboard-stats endpoint**, since several pages currently compose their view client-side from `listTenders/listDocuments` calls; a backend-driven log reusing the existing `AuditLog` table would be more efficient and accurate.
 6. **Load and performance testing around embedding throughput**, deliberately measuring behavior under a realistically large library and concurrent uploads, to know the scaling ceiling ahead of time rather than discovering it in production.
-

12. Rapid-Fire Reference

- **What ORM is used?** SQLAlchemy 2, in async mode.
- **What manages schema changes?** Alembic migrations, never auto-created tables.
- **What runs background jobs?** Celery, with Redis as broker and result backend.
- **What are Redis's three jobs here?** Celery broker, progress cache (Pub/Sub plus a snapshot key), and short-lived single-use token storage.
- **How is a long-lived token kept out of the WebSocket URL?** A short-lived, single-use ticket minted over an authenticated HTTP call, passed in the URL instead of the real access token.
- **Why GETDEL for ticket redemption?** Reads and deletes the key atomically, closing the race window a separate GET-then-DELETE would leave for a retried or double-fired connection attempt.
- **What model handles high-volume extraction calls?** A cheaper, faster Claude model (Haiku tier), via `ANTHROPIC_EXTRACTION_MODEL`.
- **What model handles the grounded Ask feature?** A stronger Claude model (Sonnet tier), via `ANTHROPIC_MODEL`.
- **What library provides embeddings, and where does it run?** `fastembed`, locally and CPU-only inside the Celery worker, no external API call.
- **What's the embedding dimension, and why can't it just change?** 768; two `vector(768)` `pgvector` columns are already committed to it, so changing it means a migration plus re-embedding everything.
- **What vector search operator is used?** Cosine distance, via `pgvector`'s `<=>` operator.
- **What are the three match confidence tiers?** Auto-matched at 0.85 or above, suggested between 0.60 and 0.84, otherwise missing.
- **What prevents a stale worker from corrupting a cancelled or retried run?** `run_generation`, checked atomically in every progress write's `WHERE` clause.
- **How many failed logins trigger a logout, by default?** Five.
- **What does login return for a wrong password versus an unknown email?** The identical generic error message, to prevent email enumeration.
- **Is there per-user ownership on tenders and documents?** No, deliberately; it's a shared team workspace, not multi-tenant.
- **What frontend routing style is used?** Classic React Router `<Routes>/<Route>` with nested layouts, not the data-router loaders/actions API.
- **What state library is used, and for what?** Zustand, for the auth store and theme store only; everything else is local or server state via hooks.
- **What's the one file allowed to call `fetch()` directly?** `src/lib/apiClient.ts`.
- **How does the frontend show a private, authenticated PDF or image?** Fetch as a blob through the authenticated client, turn it into an object URL, revoke it when done.
- **What was the plan's original frontend framework versus what was built?** Next.js was specified; the actual build is Vite plus React 19 with classic React Router, a documented and reviewed deviation.

- **What does GET /health check?** A real SELECT 1 against Postgres, not just that the process is alive.
- **What does GET /api/health/llm check?** A live round trip through the exact Anthropic client configuration real extraction uses.
- **What happens if extraction yields zero requirements?** The run is raised as a hard failure, not presented as an empty success.
- **Is the support-email SMTP feature actually used?** No, fully implemented but never called; the real support flow is a frontend mailto: link.